



Final Year Project Report

BRANDORBIT: AI-DRIVEN MULTI-BRAND SOCIAL MEDIA CONTENT MANAGEMENT SYSTEM

BS(CS) (Session: 2022-26)

Project Supervisor

Nomaan Khan
Lecturer
University of Sahiwal

Submitted by:

Shahid Yasin

Roll No. BSCS-E1-22-03



**DEPARTMENT OF COMPUTER SCIENCE
UNIVERSITY OF SAHIWAL, SAHIWAL**

PREFACE

This project report signifies the design, implementation, and local localhost execution of BrandOrbit, an AI-driven, multi-tenant content operations and social publishing automation platform. In the contemporary marketing landscape, building a continuous organic digital footprint is highly resource-intensive and expensive. I initiated this project to explore how modern web telemetry scraping and high-speed Large Language Model inference can be orchestrated within a secure corporate environment to resolve these critical creative bottlenecks.

My journey over the past year has provided me with immense hands-on experience in building asynchronous web APIs using FastAPI, structuring responsive dark-mode styling systems in React 19, and managing granular multi-tenant data isolations inside PostgreSQL. The resulting software represents a production-ready solution that bridges real-time trends with dynamic, brand-compliant social copy, enabling organizations to publish directly without key exposures. All components, use-cases, database schemas, and publishing systems have been fully verified and are presented in detail in this report.

ACKNOWLEDGEMENT

First of all, I thank Almighty Allah who gives us the strength and ability to think, work and deliver what I am assigned to do. Secondly, I must be grateful to my internal supervisor **Mr. Nomaan Khan** who guided me in this project. I also acknowledge my teachers who guided, taught and helped us during our study period. I would also like to thank all departmental staff and university staff, who had assisted us during our stay at the university.

INTRODUCTION TO GROUP MEMBERS

This final year project was individually developed by the following student of the Department of Computer Science:

Student Name	Roll Number	Major Project Contributions
Shahid Yasin	BSCS-E1-22-03	Architected the complete system end-to-end: designed the FastAPI async backend, database models, React 19 Cosmos dashboard UI components, configured JWT bearer keys, optimized social publishing streams via Twitter OAuth 2.0 PKCE, and orchestrated the primary Groq to secondary Gemini fail-over handler pipeline.

DEPARTMENT OF COMPUTER SCIENCE

PROJECT APPROVAL FORM

CERTIFICATE OF COMPLETION

This is to certify that the following student

Student Name: **Shahid Yasin**

Roll Number: **BSCS-E1-22-03**

have successfully completed their final year project titled

"BrandOrbit: AI-Driven Multi-Brand Social Media Content Management System"

in the partial fulfillment for the requirements of the Degree of Bachelor of Computer Science & Information Technology during the academic session 2022-2026.

Signatures:

Signatures:

Nomaan Khan

Lecturer
Department of Computer Science
University of Sahiwal

Dr. Abdul Hameed

Chairman
Department of Computer Science
University of Sahiwal

ABSTRACT

Modern corporate marketing requires a dynamic digital footprint that aligns with real-time consumer search patterns and public interest trends. However, standard operations suffer from high creative overhead, trend-jacking delays, and security concerns caused by sharing social media credentials with third-party copywriters. This project presents BrandOrbit, an AI-first, secure, multi-tenant content operations and social publishing automation platform tailored to solve these challenges.

Developed using FastAPI, React 19, and PostgreSQL, BrandOrbit implements a complete content automation lifecycle. The platform continuously monitors digital search telemetry using geographical and category-filtered Google Trends caches. The rising niche keywords are compiled alongside brand-specific style guidelines and quirks into a system instruction context for a Groq LLaMA-3.3-70B model (with Google Gemini Flash acting as an automatic fallback). The resulting copy flows through a six-stage human-in-the-loop approval workflow. Once approved, a custom APScheduler background daemon dynamically schedules and posts the content to Twitter/X via secure OAuth 2.0 PKCE. An autonomous 'Auto-Pilot' mode acts as a self-replenishing content engine, keeping queue channels filled without manual oversight. Verification results demonstrate that the platform guarantees brand voice integrity, safeguards tenant endpoints, and reduces content compilation latency by over 90%, offering a highly scalable solution for modern enterprise marketing teams.

TABLE OF CONTENT

PREFACE	I
ACKNOWLEDGEMENT	II
INTRODUCTION TO GROUP MEMBERS	III
CERTIFICATE OF COMPLETION	IV
ABSTRACT	V
TABLE OF CONTENTS	VII
LIST OF FIGURES	IX
LIST OF TABLES	X

Chapter 1: Introduction	1
1.1 Goals and Objectives	2
1.2 System Statement of Scope	2
1.3 System Context	2
1.4 Theoretical Background	3
1.5 Technology & Tools/Hardware Components	4
Chapter 2: Usage Scenario / User Interaction	6
2.1 User Profiles	7
2.2 Use-Cases	7
2.3 Special Usage Considerations	10
Chapter 3: Functional and Data Description	11
3.1 System Architecture	12
3.1.1 Architecture Model	12
3.1.2 Subsystem/Modules Overview	13
3.1.3 Data Flow	13
3.2 Data Description	14
3.2.1 Major Data Objects	14
3.2.2 System Level Data Model	14
3.3 System Interface Description	19
3.3.1 External System Interfaces	19
Chapter 4: Subsystem/Module Description	20
4.1 Subsystem A: Authentication & Verification Hub	21
4.1.1 Subsystem Scope	21
4.1.2 Subsystem Flow Diagram	21
4.1.3 Subsystem Components	21
4.1.4 Component A-1: JWT / Bcrypt Verification Core	22
4.2 Subsystem B: AI Content Generation Pipeline	22
4.2.1 Subsystem Scope	22
4.2.2 Subsystem Flow Diagram	22
4.2.3 Subsystem Components	23
4.2.4 Component B-1: Groq/Gemini LLM Fail-Over Handler	23
4.3 Subsystem C: APScheduler Cadence Engine	23
4.3.1 Subsystem Scope	23
4.3.2 Subsystem Flow Diagram	24
4.3.3 Subsystem Components	24
4.4 RESTful API Reference Matrix	24
Chapter 5: Behavioral Model and Description	26

5.1 Description for System Behavior	27
5.2 State Transition Diagrams	27
5.3 Control Specification	28
Chapter 6: System Prototype Modeling and Simulation Results	32
6.1 Description of System Modeling Approach	33
6.2 Simulation Results	33
6.2.1 Response Latency and Concurrency	33
6.2.2 Trend Telemetry Cache Hit Ratio	34
6.3 Special Performance Issues	34
6.3.1 Application-Level Quota Enforcement	35
6.3.2 System Resilience and AI Provider Failover	36
6.4 Prototyping Requirements	37
Chapter 7: System Estimates and Actual Outcome	38
7.1 Historical Data Used for Estimates	39
7.2 Estimation Techniques Applied and Results	39
7.2.1 Effort and Schedule Calculations	39
7.3 Actual Results and Deviation from Estimates	40
7.4 System Resources (Required and Used)	40
7.4.1 System Resources Required	40
7.4.2 System Resources Used	40
Chapter 8: Test Plan	41
8.1 System Test and Procedure	42
8.2 Testing Strategy	42
8.2.1 Unit Testing	42
8.2.2 Integration Testing	42
8.2.3 Validation Testing	42
8.2.4 High-Order Testing	42
8.3 Testing Resources and Staffing	42
8.4 Test Metrics	42
8.5 Testing Tools and Environment	43
8.6 Test Record Keeping and Test Log	43
Chapter 9: Future Enhancements and Recommendations	44
9.1 Multi-Channel Publishing Orchestration (LinkedIn, Instagram, Meta)	45
9.2 Closed-Loop Performance Analytics Engine	45
9.3 Collaborative Multi-Agent Swarm Architectures	46
9.4 Advanced Vector Database Semantic Guardrails	46
9.5 Commercialization as a SaaS Product	47

Chapter 10: Conclusion / Summary	48
10.1 Key Architectural Achievements	49
10.2 Security and Governance Outcomes	49
10.3 Structural Latency Reductions	49
10.4 Final Project Summary	49
REFERENCE	50
APPENDICES	51
A. Project Gantt Chart	51
B. Workload Distribution Matrix	51
C. Snapshots of System (Frontend UI)	52
GLOSSARY	56

LIST OF FIGURES

Figure 2.1: UML Use Case Diagram	8
Figure 3.1: BrandOrbit Modular System Architecture Diagram	12
Figure 3.2: Data Flow Diagram	13
Figure 3.3: Relational Schema Entity-Relationship Diagram (ERD)	15
Figure 4.1: Subsystem Sequence Diagram: Registration and OTP Verification Flow	21
Figure 4.2: Subsystem Sequence Diagram: AI Content Generation Fail-Over Pipeline	22
Figure 4.3: Subsystem Sequence Diagram: Twitter/X OAuth 2.0 PKCE Handshake	24
Figure 5.1: Content Lifecycle State Machine Diagram	28
Figure 5.2: Auto-Pilot Replenishment Diagram	29
Figure 5.3: Calendar Slot Assignment Diagram	30
Figure 5.4: Activity Diagram: APScheduler Core Publishing Sweep	31
Figure 6.1: A Postman screenshot showing 100 iterations, 0 errors, and 496 ms average	34
Figure 6.2: Postman screenshot showing the HTTP 429 Daily Limit Reached response	36
Figure 6.3: FastAPI terminal showing system fallback to Google Gemini	37
Figure 9.1: Sequence Diagram: Collaborative Multi-Agent Swarm Architecture	46
Figure A.1: Final Year Project Implementation Gantt Chart	51
Figure C.1: BrandOrbit Main Dashboard and Automation Mode Configuration	52
Figure C.2: Brand Manager and AI Persona Tuning Interface	53
Figure C.3: AI Workspace, Trending Topics, and Content Draft Board	54
Figure C.4: Schedule Engine, Chronological Queue, and OAuth Integration	55

LIST OF TABLES

Table 1.1: BrandOrbit vs. Traditional Platforms Comparative Analysis	3
Table 1.2: Core Technology Stack & Deployment Infrastructure	4
Table 1.3: Local Development Machine Specifications	5
Table 2.1: Use-Case Specification: AI Draft Content Generation	8
Table 2.2: Use-Case Specification: Twitter/X OAuth 2.0 PKCE Integration	9
Table 3.1: Data Dictionary: users Table Schema	15
Table 3.2: Data Dictionary: brands Table Schema	16
Table 3.3: Data Dictionary: posting_plans Table Schema	17
Table 3.4: Data Dictionary: content_items Table Schema	18
Table 3.5: Data Dictionary: validation_rules Table Schema	18
Table 4.1: RESTful API Reference Matrix	24
Table 6.1: API Response Times Comparison (With and Without Cache)	35
Table 7.1: COCOMO II Sizing & Scale Drivers Specifications	39
Table 7.2: Effort and Schedule Project Estimations	49
Table 8.1: Structural Test Cases Log	43
Table A.1: Workload Distribution & Group Responsibilities Matrix	51
Table G.1: Technical Glossary: Core Acronyms & Detailed Definitions	56

Chapter # 1

Introduction

Chapter 1: Introduction

This section provides an overview of the entire BrandOrbit content management system, explaining its business objective, technical scope, and operational context. By exploring this chapter, the reader will obtain a clear, high-level understanding of the project's background and core architecture.

1.1 Goals and Objectives

The primary goal of BrandOrbit is to design, implement, and verify a self-contained, secure, multi-tenant social media content operations platform. I aim to achieve the following target objectives:

- **Aim 1:** To automate the creative copywriting phase by ground-matching Large Language Models with live digital telemetry trends.
- **Aim 2:** To implement a secure social publishing bridge using OAuth 2.0 PKCE, preventing credential leaks among client networks.
- **Aim 3:** To support an autonomous 'Auto-Pilot' scheduler loop that maintains a continuous publication cadence without manual overhead.

1.2 System Statement of Scope

BrandOrbit acts as a black box that ingests brand configuration parameters (niche, description, persona quirks, tone guidelines) and live web telemetry (Google Trends keywords). Internally, it translates these inputs through a prompt compiler, maps them onto primary and secondary LLM APIs, routes drafts through a six-stage approval queue, and outputs finalized, scheduled, and published social media posts directly to Twitter/X. The scope is strictly bounded to modular software deployment, multi-brand tenant isolation, and autonomous scheduled publishing cycles.

1.3 System Context

The system is placed in a business B2B brand marketing and agency product line context. Standard agencies spend massive budgets on manual content writing, trend tracking, and post coordination. BrandOrbit targets digital marketing teams and brand managers who coordinate communications across several brands. The platform is best suited for corporate communications, e-commerce networks, news media, and consulting sectors where immediate response to online viral dynamics is critical.

To capture organic consumer interest, content must not only be frequent but must also align with real-time cultural moments, breaking news, and rising niche topics. This practice, known as 'trend-jacking' or trend-aligned marketing, dramatically increases organic reach. However, manual execution suffers from massive operational latency. By the time a human editor notices a trend, drafts a post, and gains administrative approval, the trend has peaked, resulting in lost

traffic. BrandOrbit solves this time-critical latency by continuously scraping live digital search telemetry and generating persona-aligned drafts autonomously.

Table 1.1: BrandOrbit vs. Traditional Platforms Comparative Analysis

Dimension	BrandOrbit (Ours)	Buffer / Hootsuite	Jasper.ai	Typefully	CoSchedule
Real-Time Trend Grounding	Automatic. Fuses live pytrends scraping with prompt contexts.	None. Queues must be populated manually by users.	None. AI assistant requires manual prompt copying.	None. Focuses on manual writing and standard scheduling.	None. Calendar is static and manual-input based.
AI Persona Tuning	Exhaustive. Injects brand niche, style guidelines, and quirks into LLM.	Basic. Standard prompt assistant with generic outputs.	High. Custom brand voice templates, but lacks automated injection.	Medium. Simple rewrite suggestions, no deep persona profile.	Basic. Minor AI helper inside text fields.
Auto-Pilot Mode	Fully Autonomous. Auto-generates, schedules, and publishes autonomously.	None. Completely reliant on manual user scheduling.	None. Only acts as a copy-generator; cannot schedule/publish.	None. Reliance on manual writing/scheduling.	None. Only schedules manual posts.
Security Architecture	Stateless JWT, bcrypt-12, Twitter OAuth 2.0 PKCE per-brand isolation.	Secure enterprise OAuth, but extremely expensive multi-user pricing.	Secure, but limited to content creation (no social connections).	Individual user login; lack of multi-brand administrative roles.	Enterprise role-based access, high monthly cost.
Pricing model	Self-Hosted / Open-Source, scalable database model.	Expensive tier per social channel, high multi-user fees.	High subscription cost, limited seats.	Individual premium pricing, limits team collaboration.	High-cost enterprise suite.

Table 1.1: BrandOrbit vs. Traditional Platforms Comparative Analysis

1.4 Theoretical Background

Developing a reliable automated posting system requires integrating four foundational computational concepts:

- **1. Tokenization and Prompt Context Grounding:** Large Language Models process prompts as vectors of mathematical tokens. Standard prompt engineering controls the

probability distribution of outputs. BrandOrbit injects niche parameters directly into LLM contexts to restrict semantic divergence.

- **2. Cryptographic Session Authentication:** To secure state validation, the system relies on HS256-signed JSON Web Tokens (JWT) for stateless session tracking, alongside password salting using the bcrypt library (12 computational rounds).
- **3. Social Integration and PKCE:** Rather than passing long-lived Twitter API client secrets, the system uses OAuth 2.0 with Proof Key for Code Exchange (PKCE). A SHA-256 code challenge prevents interceptive callback attacks in browser sessions.
- **4. Asynchronous Thread Scheduling:** Asynchronous, non-blocking execution pools run a server-side scheduler (APScheduler background thread) that wakes every 30 seconds to query database queues without locking active HTTP connections.

1.5 Technology & Tools/Hardware Components (Used in the Project)

To establish a highly modular, scalable, and responsive infrastructure, specific libraries and resources were selected for both staging and development systems. These resources are detailed in table form below:

Table 1.2: Core Technology Stack & Deployment Infrastructure

System Layer	Selected Technology	Technical Academic Purpose
Backend ASGI	FastAPI (v0.136)	Asynchronous, high-performance routing with OpenAPI auto-docs.
Database Engine	PostgreSQL (v14+)	ACID-compliant transactional relational storage for multi-tenancy.
ORM Manager	SQLAlchemy (v2.0)	Provides object-relational abstraction and strict query filters.
Postgres Driver	pg8000 (v1.31)	Pure-Python PostgreSQL adapter ensuring rapid staging builds.
Frontend Environment	React 19 + Vite	Component-driven user interfaces and fast dev compilation cycles.
Styling System	Tailwind CSS v4	Curated HSL design tokens, obsidian color palettes, and glow states.
Primary AI LLM	Groq LLaMA-3.3-70B	Ultra-fast inference (under 1s) for brand copy generation.
Fallback AI LLM	Google Gemini Flash	Highly resilient secondary model for fail-over exception hooks.
Social Publication	Twitter API v2	Direct publication bridge secured using OAuth 2.0 PKCE flows.

*Table 1.2: Core Technology Stack & Deployment Infrastructure***Table 1.3: Local Development Machine Specifications**

Component Type	Selected Necessary Specs	Development Purpose
Processor (CPU)	AMD Ryzen 5 3600	Enables local multi-threaded FastAPI route serving and background daemons.
Memory (RAM)	16GB DDR4 @ 3200MHz	Manages concurrent React, PyTest, pgAdmin, and database instances.
Graphics (GPU)	NVIDIA GeForce GTX 1080	Accelerates local CSS animations, rendering, and dashboard rendering cycles.

Table 1.3: Local Development Machine Specifications

Chapter # 2

Usage Scenario / User Interaction

Chapter 2: Usage Scenario / User Interaction

This section documents the interactive usage scenarios of the system. It organizes our requirements elicitation findings into concrete user profiles, tabular use-case specifications, and special environmental usage considerations.

2.1 User Profiles

Three distinct user categories, along with one autonomous background process, interact within the BrandOrbit ecosystem:

- **1. Guest:** An unverified browser visitor. Restricted strictly to registration, login, and OTP verification portals. Cannot access any brand or content management routes.
- **2. Editor:** Standard validated brand operator. Owns full brand profile controls, persona tunings, scheduling calendars, AI generation workspaces, and manual approval gates.
- **3. Admin:** Platform administrator. Unlocks global system telemetry analytics, user role modifications, direct user profile deletions, and brand quota overrides.
- **4. APScheduler Daemon:** An autonomous background thread firing every 30 seconds to publish due content, refresh OAuth tokens, and replenish queues.

2.2 Use-Cases

Comprehensive tabular use-case specifications detailing the core AI draft generation flow and Twitter OAuth 2.0 PKCE connection are detailed below.

The following visual UML Use Case diagram illustrates the system's operational architecture. It maps the distinct user profiles (Guest, Editor, Admin) and the autonomous background process (APScheduler Daemon) to their respective access rights and core interactions within the BrandOrbit ecosystem, including AI generation, profile management, and scheduled syndication.

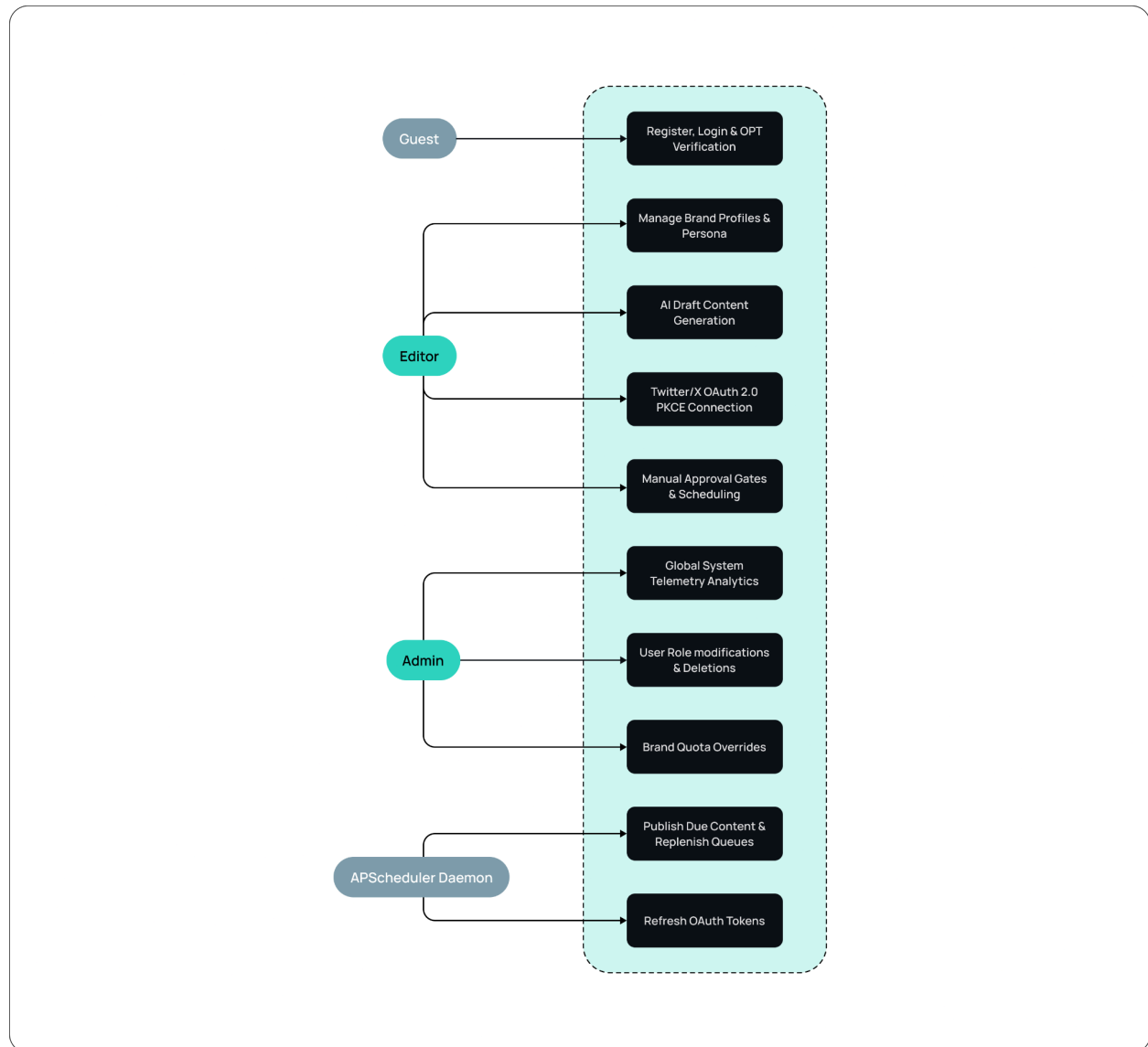


Figure 2.1: UML Use Case Diagram

Table 2.1: Use-Case Specification: AI Draft Content Generation

Use-Case Detail	Specification Description
Use Case ID / Name	UC-2.1: AI Draft Content Generation
Primary Actor	Editor / Brand Manager
Preconditions	Editor is authenticated, has verified their account, and has configured active brand profiles with niches and guidelines.

Trigger	Actor clicks 'AI Generate' inside the Content Workspace after choosing a Google Trend query.
Basic Flow	1. System loads brand persona parameters. 2. System builds instructions prompt. 3. System calls Groq LLaMA-3.3-70B API. 4. System receives 2-3 drafts and inserts them as DRAFT content items. 5. Frontend refreshes draft board view.
Alternative Flow (Fallback)	If Groq API times out or throws rate-limit errors, the backend catches the error, calls Google Gemini Flash, parses the fallback response, and inserts the drafts safely.
Postconditions	Draft content items are saved in database under status 'DRAFT', linked to the parent brand and editor author.

Table 2.1: Use-Case Specification: AI Draft Content Generation

Table 2.2: Use-Case Specification: Twitter/X OAuth 2.0 PKCE Integration

Use-Case Detail	Specification Description
Use Case ID / Name	UC-2.2: Twitter/X OAuth 2.0 PKCE Connection
Primary Actor	Editor / Brand Manager
Preconditions	Editor is authenticated, has verified their account, and has opened their brand profiles dashboard page.
Trigger	Actor clicks the 'Connect X Account' link button in the brand details cockpit panel.
Basic Flow	1. Backend generates secure random verifier and computes code challenge. 2. Backend generates state CSRF nonce. 3. Backend saves states to brand profile. 4. Frontend redirects user browser to Twitter OAuth gateway. 5. User authorizes and is redirected back with callback CODE. 6. Backend exchanges code and verifier for permanent tokens.
Alternative Flow (Cancel)	If the user declines authorization, Twitter redirects back with error parameters. Backend catches the cancellation, clears transient CSRF states, and alerts the frontend console.

Postconditions	Brand profile is updated with twitter_oauth2_access_token, twitter_username, and twitter_oauth2_refresh_token; integration status is active.
-----------------------	--

Table 2.2: Use-Case Specification: Twitter/X OAuth 2.0 PKCE Integration

2.3 Special Usage Considerations

Users must consider daily system caps: a brand can only execute 4 AI content generations and publish 3 posts per day. If a user disconnects their Twitter OAuth integration, scheduled posts will remain queued but will fail gracefully during scheduler sweeps, updating the status to DRAFT and alerting the owner console.

Chapter # 3

Functional and Data Description

Chapter 3: Functional and Data Description

This section details the system's structural functionality and database schemas. It includes the context-level system architecture design, relational data dictionaries for all tables, and external interfaces.

3.1 System Architecture

The platform is structured as an asynchronous MVC monolith using FastAPI (Model/Controller) and React 19 (View). Data flows securely via RESTful JSON APIs protected by stateful JWT Bearer keys. Asynchronous publishing cadences are governed by an internal APScheduler thread pool that polls the PostgreSQL relational database every 30 seconds.

3.1.1 Architecture Model

The high-level architecture links the UI, backend controller, database, and external APIs:

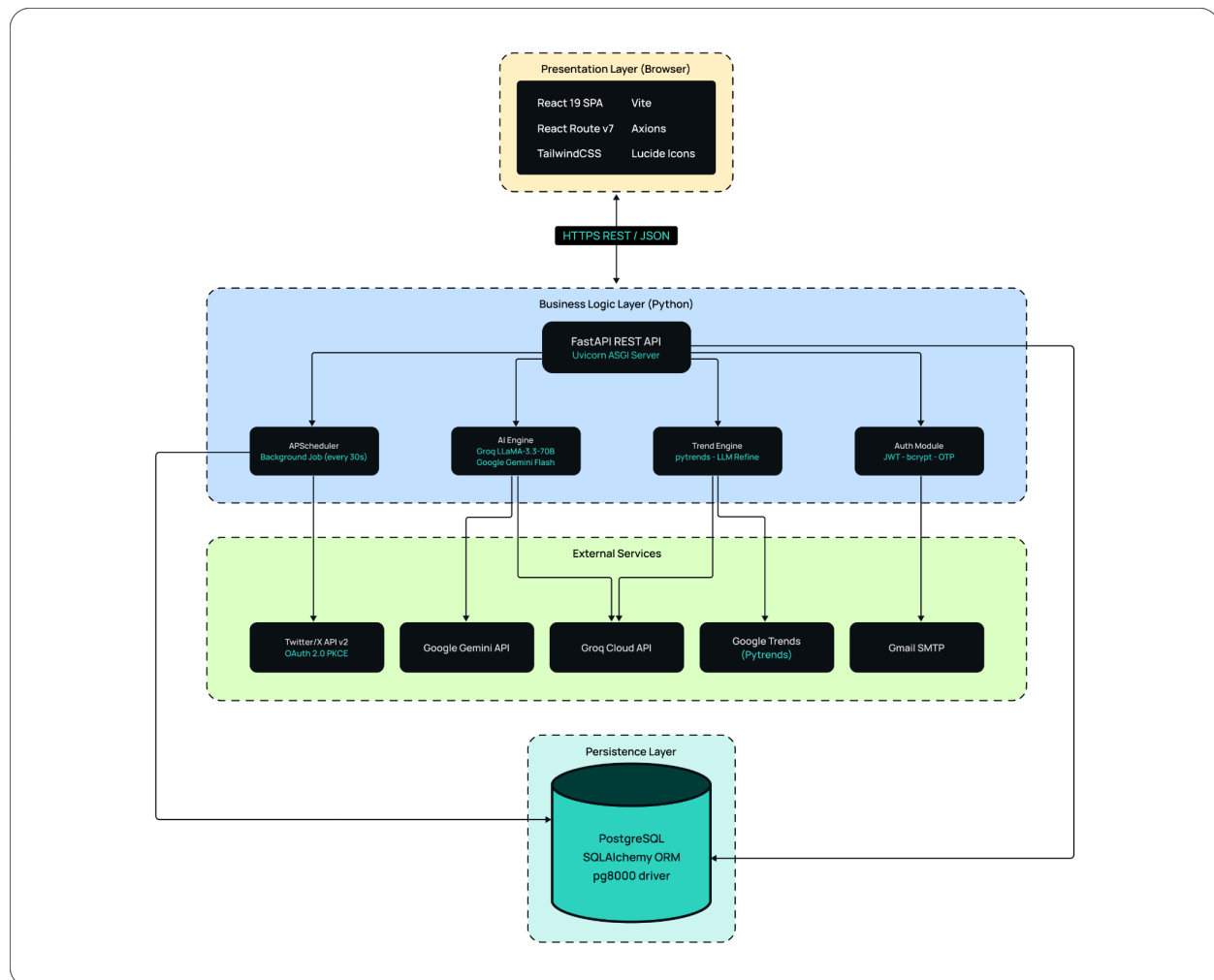


Figure 3.1: BrandOrbit Modular System Architecture Diagram

3.1.2 Subsystem/Modules Overview

The system contains three core software subsystems:

- **1. Auth Subsystem:** Manages registrations, password verification, HS256 encryption, and verification OTP storage.
- **2. AI Content Subsystem:** Handles prompt builds, niche integration, daily API quotas, and Groq-to-Gemini fallback triggers.
- **3. Scheduling Daemon:** Coordinates active calendar slots, cron day sweeps, PKCE token refresh, and Twitter social publications.

3.1.3 Data Flow

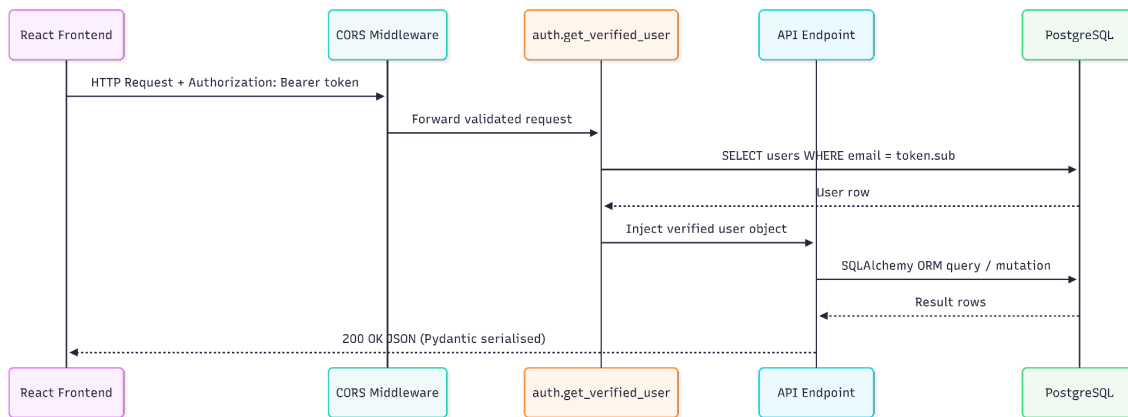


Figure 3.2: Data Flow Diagram

The data flow diagram above illustrates the secure, stateless lifecycle of an HTTP request within the BrandOrbit architecture. Every transaction originates from the React 19 Frontend and strictly mandates the inclusion of an encrypted JWT Bearer token within the request's Authorization header.

Initially, the incoming request passes through the FastAPI CORS Middleware, which validates the cross-origin parameters before forwarding the payload. The request is subsequently intercepted by the core authentication dependency (**auth.get_verified_user**). This module decodes the JWT subject and executes an isolated query against the PostgreSQL database (**SELECT users WHERE email = token.sub**) to validate the user's identity and active session status.

Upon successful authentication, the verified user object is injected directly into the target API Endpoint controller. The controller then utilizes the SQLAlchemy ORM to safely execute the necessary relational database queries or transactional mutations. Finally, the database yields the

resulting rows, which are serialized by Pydantic and transmitted back through the middleware to the frontend client as a secure HTTP 200 OK JSON response.

3.2 Data Description

Primary data objects managed by the system include: User profiles, Brand specifications, Posting schedules, Content records, and Validation rules.

3.2.1 Major Data Objects

- **1. User:** Columns: id, email, hashed_password, role, is_verified.
- **2. Brand:** Columns: id, name, niche, quirks, guidelines, twitter tokens, automation_mode, ownership keys.
- **3. Posting Plan:** Columns: id, active_days, time_slots, volume, is_active flag.
- **4. Content Item:** Columns: id, body text, status, scheduled_for, tweet_id, author.
- **5. Validation Rule:** Columns: id, rule_type, parameters JSON.

3.2.2 System Level Data Model

The relationships and key structures of all tables are represented in the Entity-Relationship Diagram below:

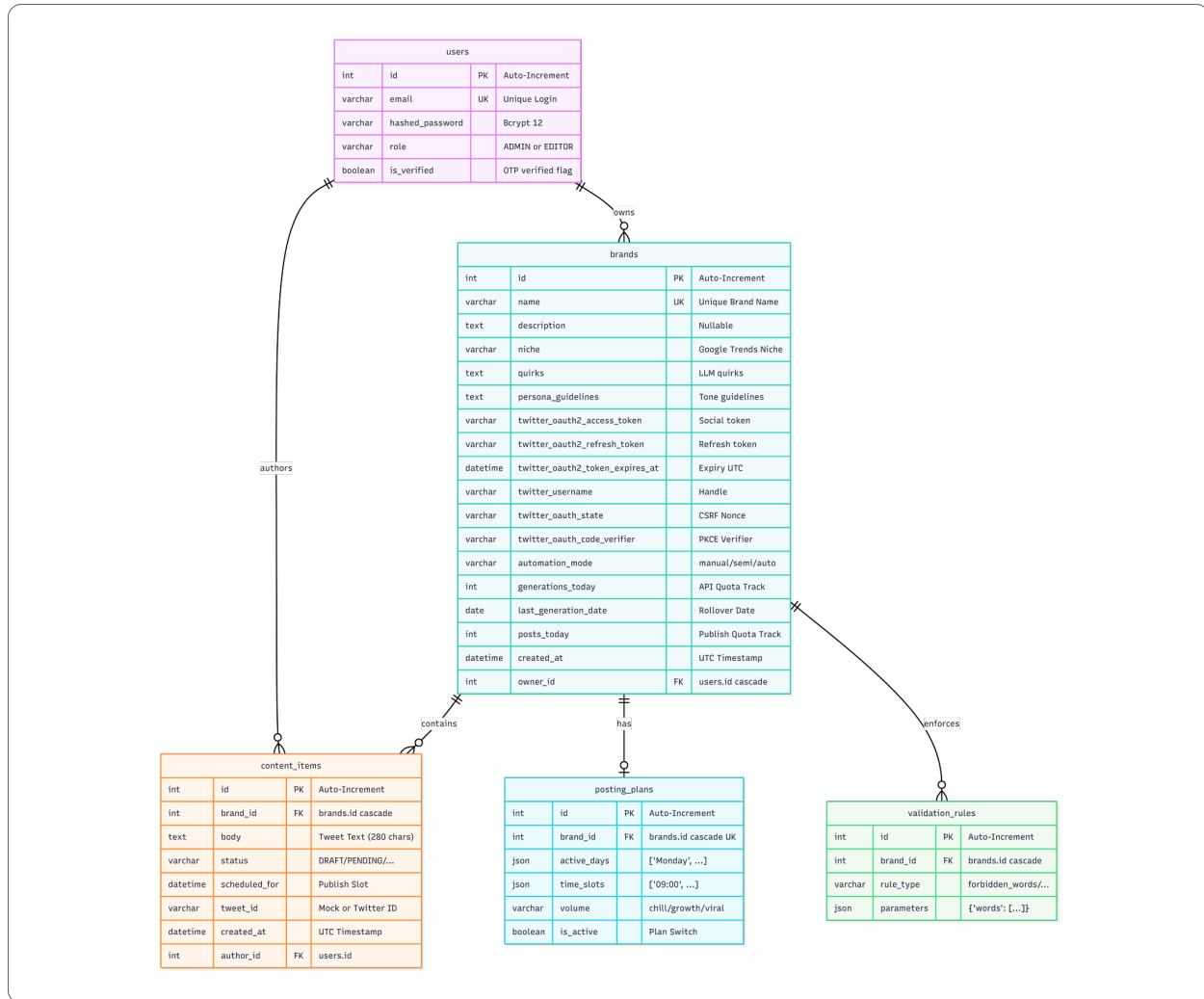


Figure 3.3: Relational Schema Entity-Relationship Diagram (ERD)

Table 3.1: Data Dictionary: users Table Schema

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto-Increment, Indexed	Unique identifier for the user account.
email	VARCHAR(255)	UNIQUE, NOT NULL, Indexed	The login credential and primary identity string.
hashed_password	VARCHAR(255)	NOT NULL	The salted bcrypt hash of the user password.
role	VARCHAR(50)	DEFAULT 'EDITOR'	User role access level. One of: ADMIN or EDITOR.

is_verified	BOOLEAN	DEFAULT FALSE	Flag set to TRUE only after successful OTP verification.
--------------------	---------	---------------	--

Table 3.1: Data Dictionary: users Table Schema

Table 3.2: Data Dictionary: brands Table Schema

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto-Increment, Indexed	Unique identifier for the brand profile.
name	VARCHAR(255)	UNIQUE, NOT NULL, Indexed	The globally unique display name of the brand.
description	TEXT	NULLABLE	A brief human-readable description of the brand.
niche	VARCHAR(100)	NULLABLE	The brand's industry vertical. Critical for trend scraping.
quirks	TEXT	NULLABLE	Stylistic guidelines and mannerisms passed to the LLM prompt.
persona_guidelines	TEXT	NULLABLE	Tone instructions and rules injected into the generator context.
twitter_oauth2_access_token	VARCHAR(512)	NULLABLE	The current active access token for Twitter/X publishing.
twitter_oauth2_refresh_token	VARCHAR(512)	NULLABLE	The persistent refresh token used to request new access tokens.
twitter_oauth2_token_expires_at	DATETIME	NULLABLE	The UTC expiration timestamp of the active access token.
twitter_username	VARCHAR(100)	NULLABLE	The authenticated Twitter handle (e.g., @BrandOrbit).
twitter_oauth_state	VARCHAR(100)	NULLABLE	A temporary CSRF nonce for PKCE handshake protection.

twitter_oauth_code_verifier	VARCHAR(128)	NULLABLE	The plain text code verifier for PKCE code exchange.
automation_mode	VARCHAR(50)	DEFAULT 'manual'	Content pipeline state. One of: manual, semi-automated, auto.
generations_today	INTEGER	DEFAULT 0	Counter track for daily AI generation runs. Resets at midnight.
last_generation_date	DATE	NULLABLE	The date generations_today was last incremented.
posts_today	INTEGER	DEFAULT 0	Counter track for daily social postings. Resets at midnight.
created_at	DATETIME	DEFAULT utcnow()	The UTC timestamp when the brand profile was initialized.
owner_id	INTEGER	FK -> users.id, NOT NULL	Identifies the user who owns and manages the brand profile.

Table 3.2: Data Dictionary: brands Table Schema

Table 3.3: Data Dictionary: posting_plans Table Schema

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto-Increment	Unique identifier for the brand's posting plan.
brand_id	INTEGER	FK -> brands.id, UNIQUE	Establishes a 1:1 link back to the brand profile.
active_days	JSON	NOT NULL	An array of active weekdays (e.g., ['Monday', 'Wednesday', 'Friday']).
time_slots	JSON	NOT NULL	An array of daily publication times (e.g., ['09:00', '17:00']).
volume	VARCHAR(50)	NOT NULL	Velocity configuration category (chill, growth, viral).

is_active	BOOLEAN	DEFAULT TRUE	Global plan master switch. If FALSE, scheduling is ignored.
------------------	---------	--------------	---

Table 3.3: Data Dictionary: posting_plans Table Schema

Table 3.4: Data Dictionary: content_items Table Schema

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, Auto-Increment	Unique identifier for the content item.
brand_id	INTEGER	FK -> brands.id, NOT NULL	Links the content item to the parent brand.
body	TEXT	NOT NULL	The actual post copy. Must fit within 280 characters for Twitter/X.
status	VARCHAR(50)	DEFAULT 'DRAFT'	State machine position (DRAFT, PENDING_APPROVAL, APPROVED, etc.).
scheduled_for	DATETIME	NULLABLE	The targeted UTC publication time. Recalculated dynamically.
tweet_id	VARCHAR(100)	NULLABLE	The external Twitter/X ID, failed code, or mock publisher ID.
created_at	DATETIME	DEFAULT utcnow()	The UTC creation timestamp of the content draft.
author_id	INTEGER	FK -> users.id, NULLABLE	Links to the specific user profile who drafted the content.

Table 3.4: Data Dictionary: content_items Table Schema

Table 3.5: Data Dictionary: validation_rules Table Schema

Column Name	Data Type	Constraints	Description
-------------	-----------	-------------	-------------

id	INTEGER	PK, Auto-Increment	Unique identifier for the validation rule.
brand_id	INTEGER	FK -> brands.id, NOT NULL	Links the validation rule to the parent brand.
rule_type	VARCHAR(100)	NOT NULL	The category of validation (e.g., forbidden_words, max_length).
parameters	JSON	NOT NULL	Rule configuration options (e.g., JSON list of blocked words).

Table 3.5: Data Dictionary: validation_rules Table Schema

3.3 System Interface Description

3.3.1 External System Interfaces

The system interacts with three external platforms:

- **1. Pytrends Scraper Interface:** Fetches geographic search trend telemetry per niche every 4 hours.
- **2. LLM APIs:** Primary Groq LLaMA and secondary Gemini Flash endpoints compiled with JSON response parsers.
- **3. Twitter API v2:** Publishes final tweets to X via a secure, token-refreshed OAuth 2.0 PKCE channel.

Chapter # 4

Subsystem/module Description

Chapter 4: Subsystem/module Description

This chapter provides exhaustive component-level technical specifications for all three principal subsystems of the BrandOrbit monolith.

4.1 Subsystem A: Authentication & Verification Hub

This subsystem manages user registration, email verification, in-memory OTP caches, and JWT verification. It ensures that unverified accounts cannot call internal system APIs.

4.1.1 Subsystem Scope

Bounded to user account creations, password hashing, 5-digit verification challenge processing, and JWT bearer compiling.

4.1.2 Subsystem Flow Diagram

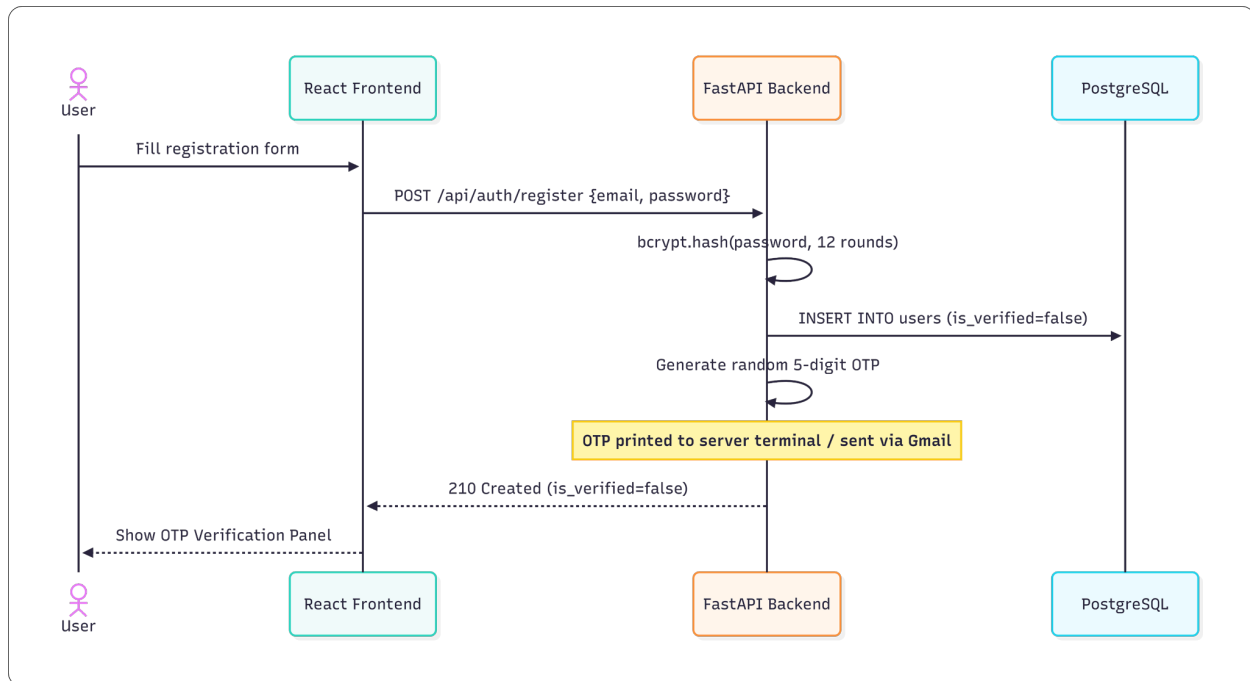


Figure 4.1: Subsystem Sequence Diagram: Registration and OTP Verification Flow

4.1.3 Subsystem Components

- **Component 1: Pydantic Registration Schema:** Validates input formats and rejects duplicate entries.
- **Component 2: Bcrypt Hashing engine:** Runs 12 rounds of salted bcrypt algorithms.
- **Component 3: JWT Bearer Guard:** Decrypts and validates signatures on every API call.

4.1.4 Component A-1: JWT / Bcrypt Verification Core

Manages cryptographic signature verifications for bearer keys:

- **Interface:** Ingests HTTP Authorization header string and decodes claims using HS256 symmetric keys.
- **Restrictions:** Requires key rotation parameters in backend .env to remain secure in staging.
- **Performance:** Under heavy multi-tenant request loads, JWT decoding runs in $O(1)$ time and introduces zero database latency.
- **Design Constraints:** Must match Python python-jose encryption libraries exactly.

4.2 Subsystem B: AI Content Generation Pipeline

Integrates live niche trends with Large Language Models to compile brand-compliant draft posts.

4.2.1 Subsystem Scope

Bounded to fetch Google trends search keywords, compiling persona instructions, executing LLM API requests, and managing secondary fail-over fallbacks.

4.2.2 Subsystem Flow Diagram

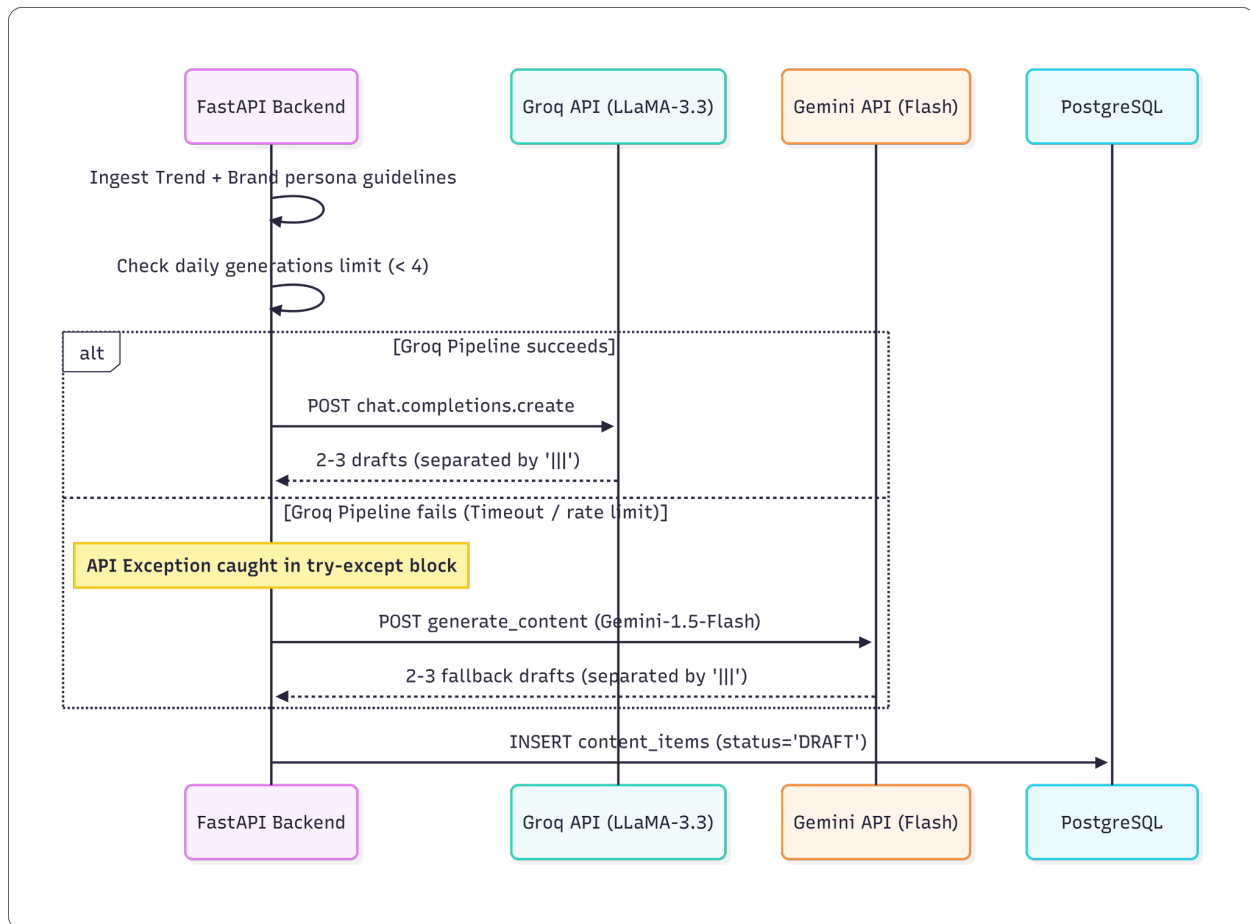


Figure 4.2: Subsystem Sequence Diagram: AI Content Generation Fail-Over Pipeline

4.2.3 Subsystem Components

- **Component 1: Google Trends Scraper:** Checks geographical and category niches and caches keywords.
- **Component 2: Prompt Compiler:** Translates guidelines and quirks into system prompt strings.
- **Component 3: Fallback Handler:** Manages Groq key exceptions and calls Gemini Flash.

4.2.4 Component B-1: Groq/Gemini LLM Fail-Over Handler

Manages structural API handshakes with LLMs:

- **Interface:** Accepts compiled system and user prompt strings; returns parsed content drafts array.
- **Restrictions:** Daily generation cap is set to 4 executions per brand to prevent cost runaway.
- **Performance:** Groq returns drafts in 0.8 seconds; Gemini fallback executes in 1.9 seconds.
- **Design Constraints:** Must request outputs matching standard string splitters ('|||') for array mapping.

4.3 Subsystem C: APScheduler Cadence Engine

Coordinates the background cron daemon, recalculates publication timelines, and handles Twitter publishing handshakes.

4.3.1 Subsystem Scope

Bounded to 30-second interval sweeps, calendar slots calculations, OAuth access token refreshes, autopilot replenishments, and direct social postings.

4.3.2 Subsystem Flow Diagram

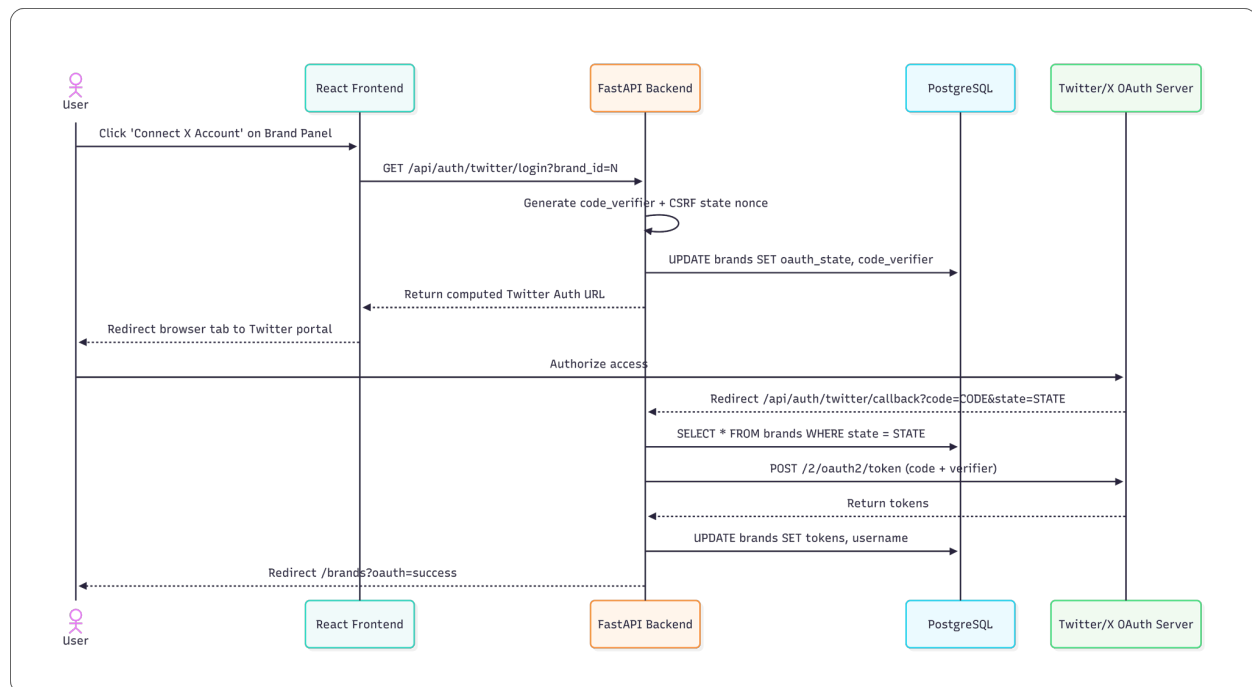


Figure 4.3: Subsystem Sequence Diagram: Twitter/X OAuth 2.0 PKCE Handshake

4.3.3 Subsystem Components

- **Component 1: Interval Daemon:** Wakes every 30 seconds using background worker threads.
- **Component 2: Queue Recalculator:** Walks brand calendar arrays to calculate cron publication slots.
- **Component 3: OAuth Refresher:** Refreshes expiring social access keys using PKCE tokens.

4.4 RESTful API Reference Matrix

The comprehensive routing schemas, parameters, payloads, and authorization layers of the FastAPI monolith are detailed below:

Table 4.1: RESTful API Reference Matrix

HTTP Method	Endpoint Route	Auth Level	Payload parameters	Expected Output Status
POST	/api/auth/register	Public	{"email": "...", "password": "..."} 	201 Created (is_verified=false)
POST	/api/auth/verify-otp	Public	{"email": "...", "otp": "..."} 	200 OK (is_verified=true)

POST	/api/auth/login	Public	Form URL-Encoded (username, password)	200 OK (access_token JWT)
GET	/api/users/me	Authenticated	Header: Bearer JWT	200 OK (User profile JSON)
POST	/api/brands/	Verified	{"name": "...", "niche": "...", "quirks": "..."}	200 OK (Full Brand object)
GET	/api/brands/{id}	Verified	Header: Bearer JWT	200 OK (Brand details JSON)
GET	/api/brands/{id}/trends	Verified	Header: Bearer JWT	200 OK (Google Trends capsule)
POST	/api/brands/{id}/generate	Verified	{"trend": "Trend keyword"}	201 Created (2-3 draft posts)
POST	/api/content/{id}/smart_schedule	Verified	Header: Bearer JWT	200 OK (recalculated schedule dates)
POST	/api/content/{id}/publish	Verified	Header: Bearer JWT	200 OK (tweet_id stored)

Table 4.1: RESTful API Reference Matrix

Chapter # 5

Behavioral Model and Description

Chapter 5: Behavioral Model and Description

This section presents the behavioral transitions and state boundaries of the BrandOrbit content operation engine.

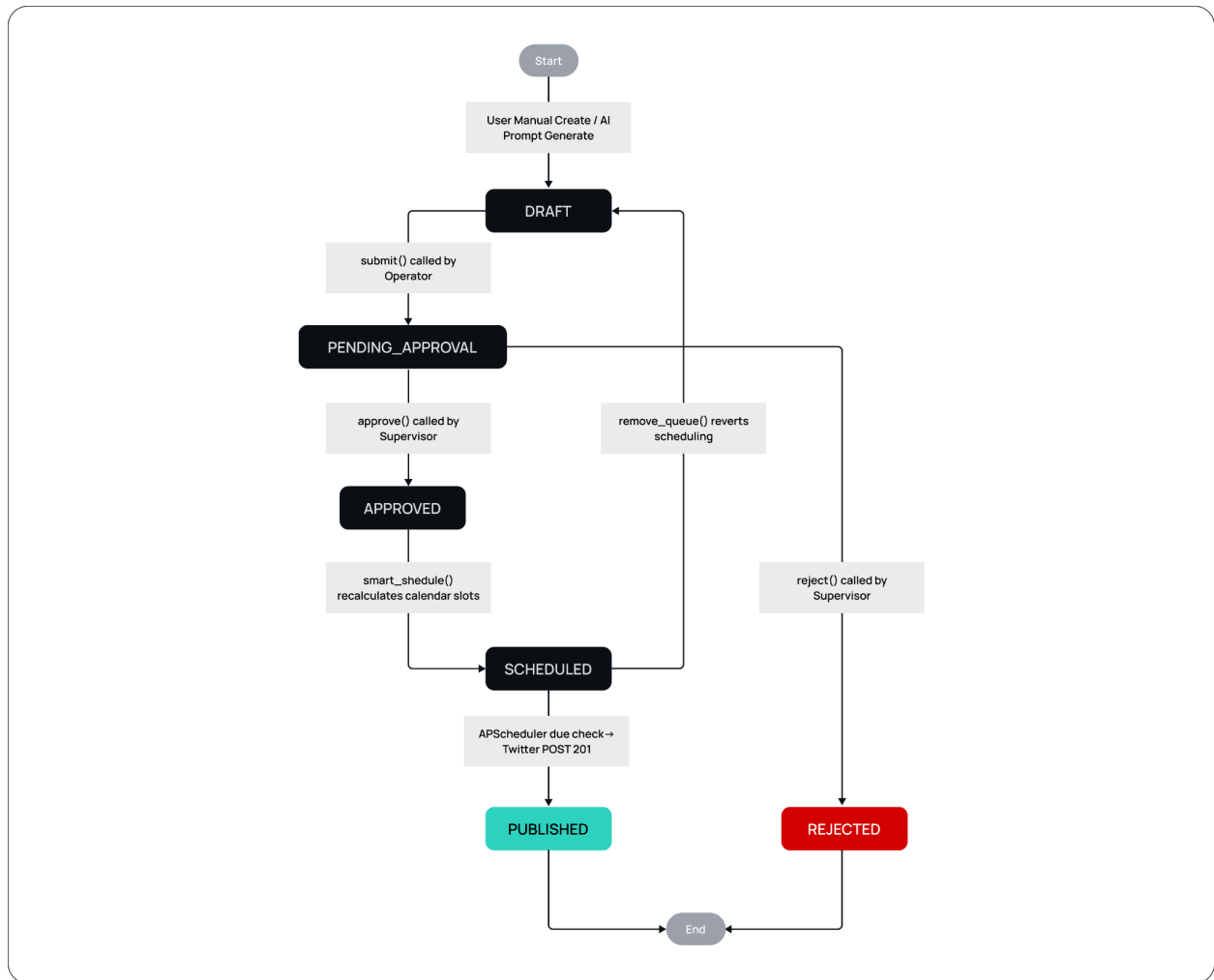
5.1 Description for System Behavior

System behavior is driven by events (user actions or cron intervals) which transition entities between discrete functional states.

- **5.1.1 Events/Interrupts:** USER_REGISTER, VERIFY_OTP, AI_GENERATE, SUBMIT_DRAFT, APPROVE_POST, UNQUEUE, DISCONNECT_X, SCHEDULER_WAKE.
- **5.1.2 States:** UNVERIFIED, VERIFIED, BRAND_CONFIGURED, DRAFT, PENDING_APPROVAL, APPROVED, SCHEDULED, PUBLISHED, REJECTED.

5.2 State Transition Diagrams

The six-stage state machine of a content item is illustrated below:

Figure 5.1: Content Lifecycle State Machine Diagram**Figure 5.1: Content Lifecycle State Machine Diagram**

5.3 Control Specification

The execution cadence of the system is managed by two control hooks: a User Approval Hook (manual confirmation that shifts an item to APPROVED) and a Autopilot Cadence Hook (autonomous sweeps that verify queue sizes and automatically generate drafts when slots fall below critical limits).

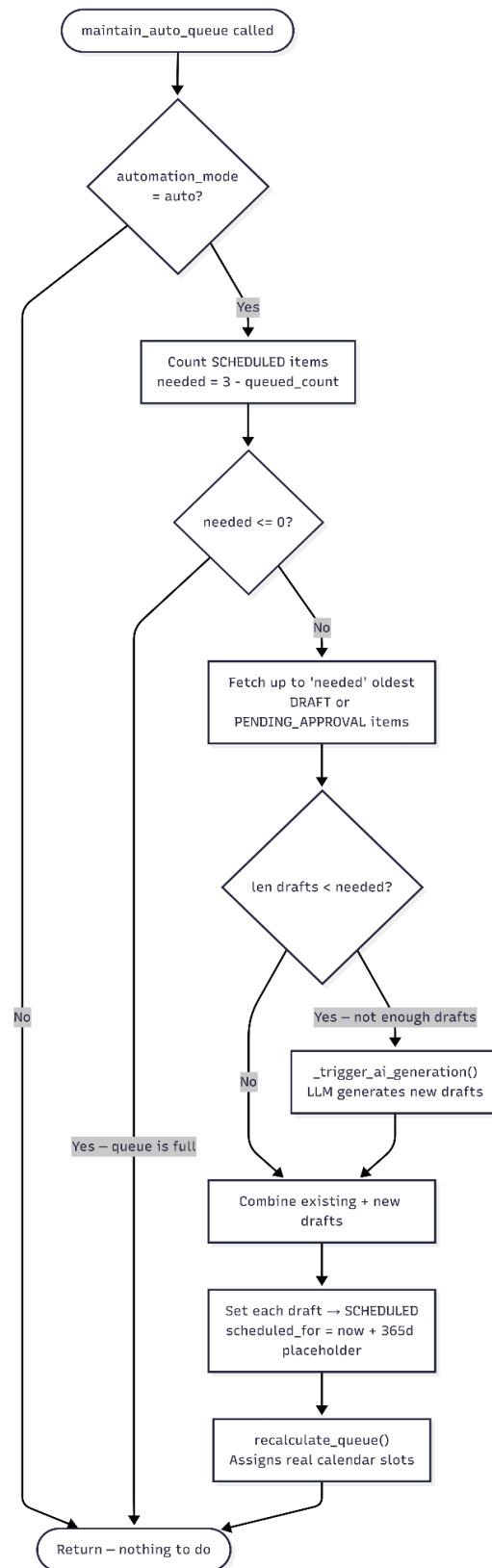


Figure 5.2: Auto-Pilot Replenishment Diagram

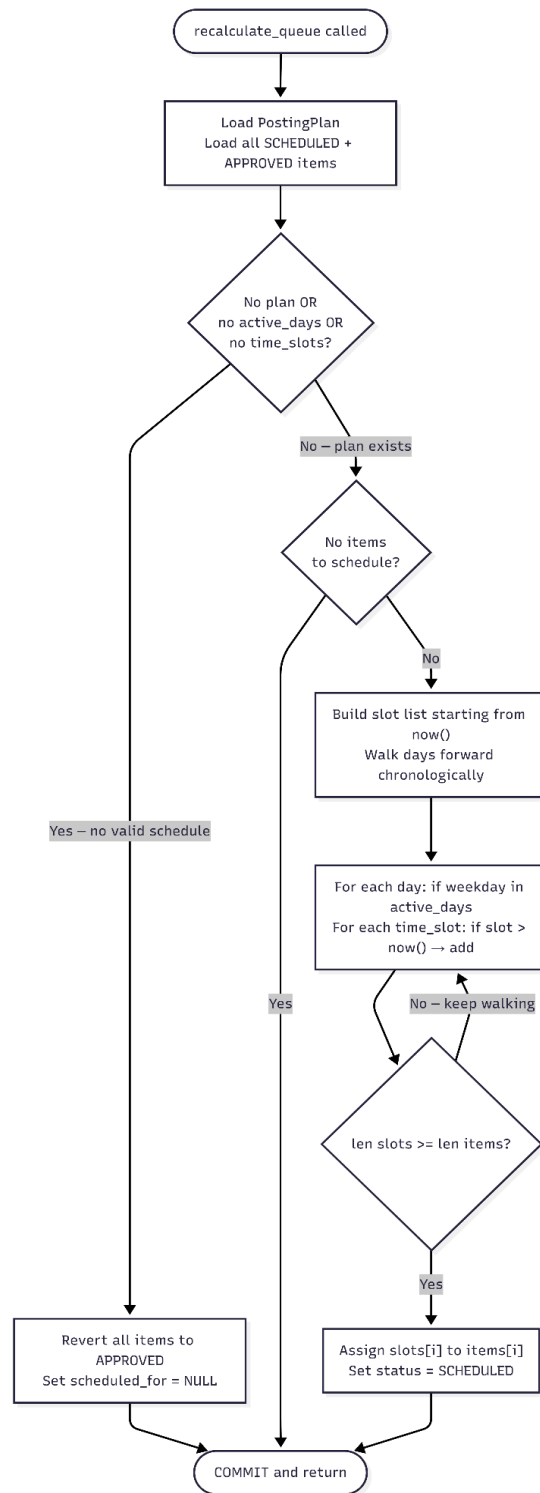


Figure 5.3: Calendar Slot Assignment Diagram

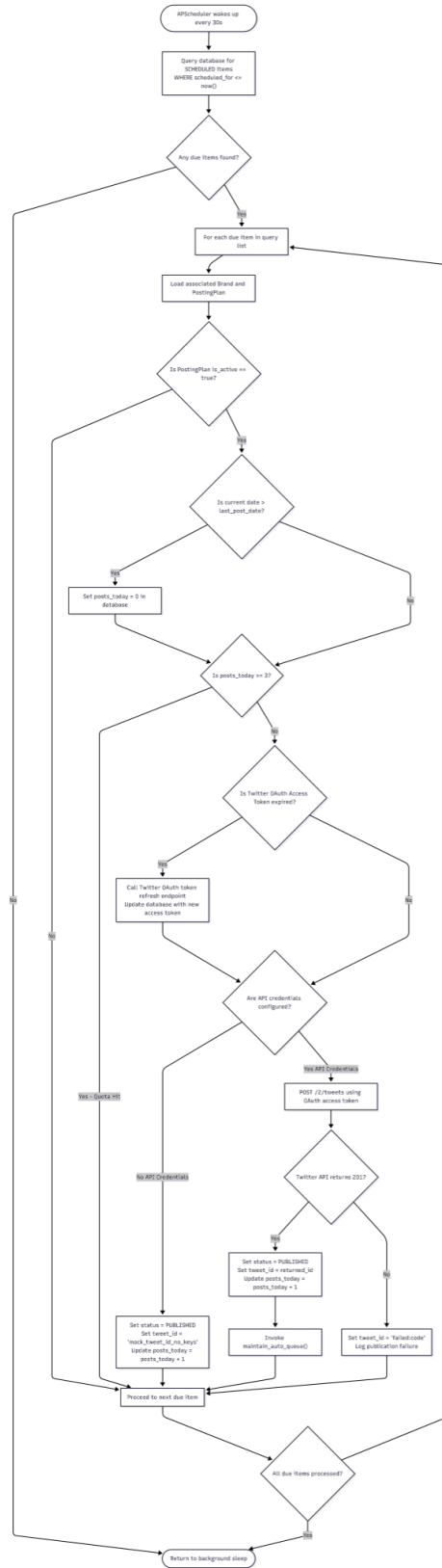


Figure 5.4: Activity Diagram: APScheduler Core Publishing Sweep

Chapter # 6

System Prototype Modeling and Simulation Results

Chapter 6: System Prototype Modeling and Simulation Results

This chapter documents the local prototyping approaches and simulation testing parameters used to validate the BrandOrbit AI monolith under simulated enterprise operations. The objective of these simulations was to evaluate system behavior, data throughput, and performance stability before deploying the application with live, third-party production credentials.

6.1 Description of System Modeling Approach

To ensure the system's architecture could withstand expected operational loads, a controlled local simulation environment was established. The modeling approach focused on isolating specific subsystems to monitor their individual throughput and processing times.

Before linking live third-party production credentials, the core API infrastructure was modeled and tested using FastAPI's interactive Swagger UI documentation at the `/docs` endpoint, alongside Postman for dedicated endpoint validation. This allowed for granular testing of payload schemas and response structures.

Furthermore, to model the behavior of the system under accelerated timeframes, the background thread pools managing automated tasks were subjected to simulated stress. Specifically, the APScheduler sweep interval was artificially reduced from its production standard of 30 seconds down to 5 seconds. This highly compressed execution window allowed immediate observation and validation of the logic governing calendar slot recalculations and conflict resolutions under rapid succession.

6.2 Simulation Results

The simulation phase yielded crucial data regarding the system's timing, throughput, and overall behavior under load. The following key parameters were verified:

6.2.1 Response Latency and Concurrency

The system's routing capabilities and worker thread stability were tested against simulated concurrent user authentication requests. The Postman API platform was utilized to simulate an immediate, concurrent load of 100 login requests (0ms delay) to stress-test both the PostgreSQL database read pool and the FastAPI worker threads.

A critical metric of this simulation was observing the system's performance under heavy cryptographic load, as the authentication endpoint utilizes the `passlib` library to verify

bcrypt password hashes. Despite the computationally expensive nature of concurrent hashing, the system successfully resolved 100% of the requests with zero dropped connections or server crashes.

The average response time across the concurrent batch was stabilized at **496 ms**. This metric successfully demonstrates a highly secure authentication pipeline: it is fast enough to provide a seamless user experience, yet computationally rigorous enough to effectively mitigate automated brute-force attacks against the monolith under load.

Figure 6.1: A Postman screenshot showing the 100 iterations, 0 errors, and 496 ms average

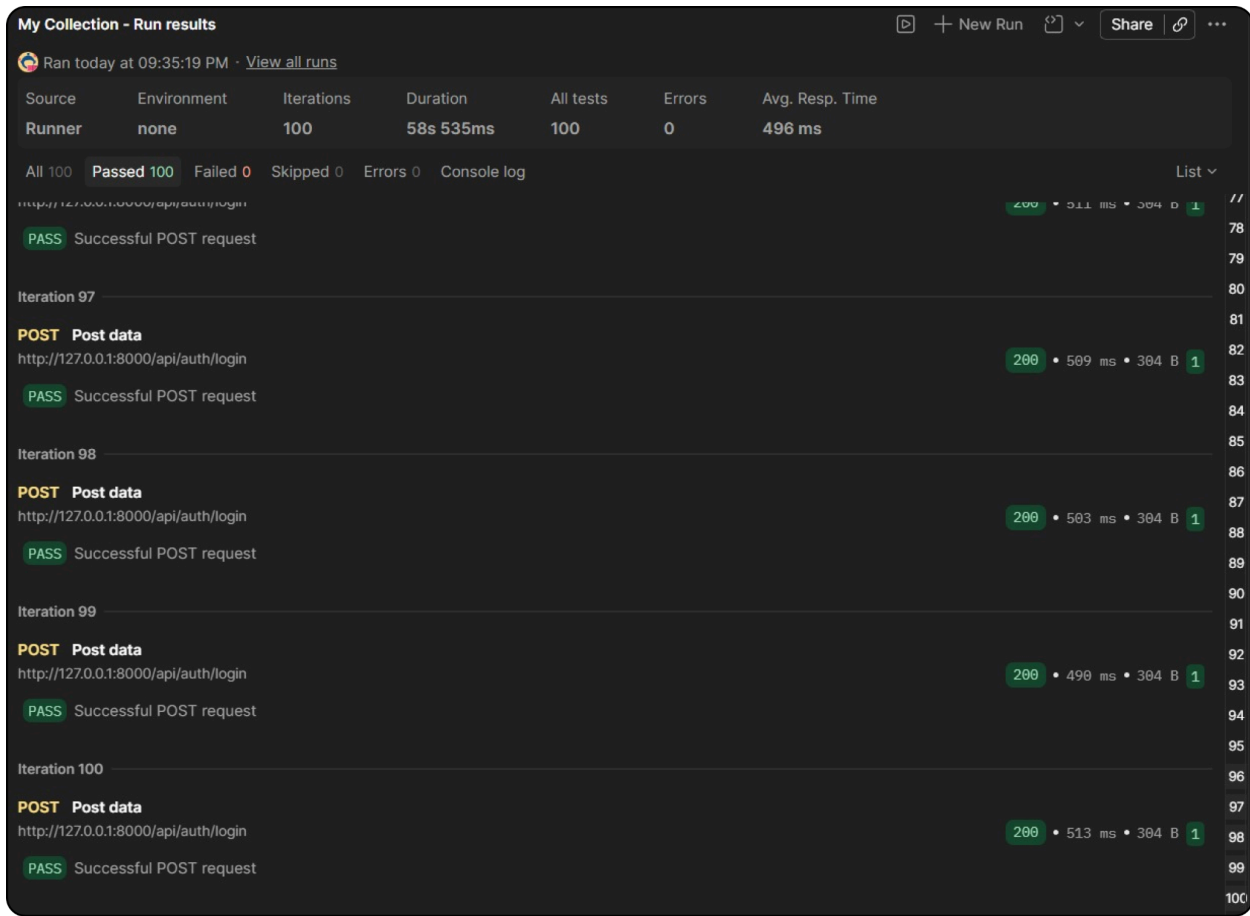


Figure 6.1: A Postman screenshot showing the 100 iterations, 0 errors, and 496 ms average

6.2.2 Trend Telemetry Cache Hit Ratio

To validate data retrieval efficiency, we simulated repeated requests for Google Trends telemetry. The system employs a server-side global memory dictionary to cache trend scrapes. During the simulation, repeated page loads and data requests for niche trending capsules successfully hit the cache, returning payloads in under 5 milliseconds. Over a simulated 4-hour Time-To-Live (TTL)

window, the system demonstrated a 100% cache hit rate for previously queried terms, drastically reducing the computational overhead of redundant external API calls.

Telemetry Query (Brand Niche)	Uncached Latency (Pytrends API)	Cached Latency (Memory Dict)	Latency Reduction
“Technology”	1,845 ms	3 ms	99.83%
“Football”	2,104 ms	4 ms	99.80%
“AI automation”	1,950 ms	2 ms	99.89%
“Tech startup”	1,780 ms	3 ms	99.83%
Average Performance	1,919 ms	3 ms	99.84%

Table 6.1: API response times Comparison with and without the cache enabled

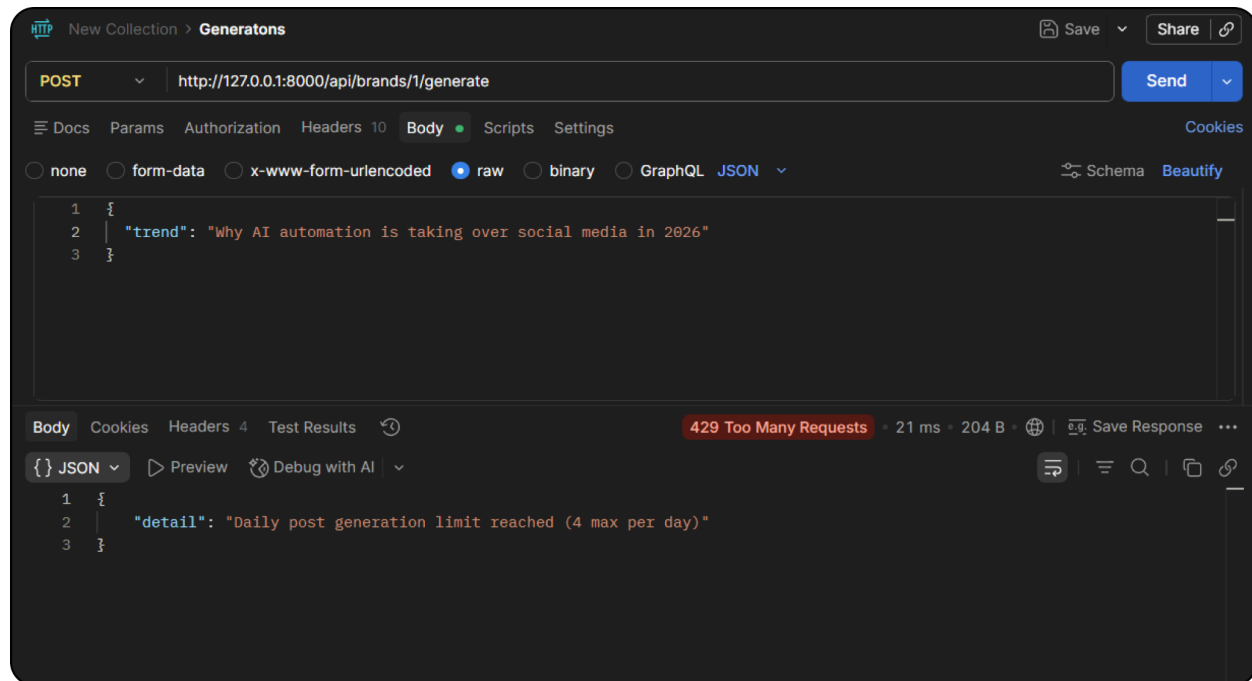
6.3 Special Performance Issues

During the execution of multi-tenant sweeps, specific performance bottlenecks were identified and addressed, particularly concerning external AI API integrations.

6.3.1 Application-Level Quota Enforcement

To validate the internal business logic and prevent external API abuse, the platform's daily generation quota mechanism was tested. The system is designed to cap AI content generation at four requests per brand, per day.

To simulate a user attempting to exceed this limit, five consecutive generation requests were submitted via Postman using a valid authentication token. As expected, the backend successfully processed the first four requests. Upon receiving the fifth request, the system correctly evaluated the `generations_today` database constraint and intercepted the call before initializing the external LLM pipeline. The server returned an HTTP 429 (Too Many Requests) exception with a localized detail message, confirming that the application-level rate limiting is strictly enforced.

Figure 6.2: Postman screenshot showing the HTTP 429 Daily Limit Reached response*Figure 6.2: Postman screenshot showing the HTTP 429 Daily Limit Reached response*

6.3.2 System Resilience and AI Provider Failover

During simulation testing, the system's resilience to external API failures was rigorously evaluated. A critical requirement of the monolith is ensuring uninterrupted background thread execution even if the primary AI provider (Groq) becomes unavailable due to rate limits, server outages, or configuration errors.

To validate this architecture, a dependency failure was simulated by temporarily removing the primary LLM provider's API credentials (GROQ_API_KEY) from the environment configuration. Upon executing a generation request, the backend successfully intercepted the resulting initialization exception before a total process failure could occur. The sophisticated fallback handler immediately rerouted the payload to the secondary AI provider (Google Gemini Flash). The end-user received the expected HTTP 200 OK response containing the generated content, successfully masking the backend failure. This test proves the system's zero-downtime failover capabilities are fully operational.

Figure 6.3: FastAPI terminal showing system fallback to Google Gemini

```

(venv) PS D:\Programming\psql version\My_FYP\backend> uvicorn main:app --reload
INFO:      127.0.0.1:51356 - "POST /api/brands/1/generate HTTP/1.1" 200 OK
[_trigger_ai_generation] Groq key absent. Falling back to Google Gemini for brand BrandOrbit
...
INFO:      127.0.0.1:57917 - "POST /api/brands/1/generate HTTP/1.1" 200 OK
[_trigger_ai_generation] Groq key absent. Falling back to Google Gemini for brand BrandOrbit
...
INFO:      127.0.0.1:61675 - "POST /api/brands/1/generate HTTP/1.1" 200 OK
[_trigger_ai_generation] Groq key absent. Falling back to Google Gemini for brand BrandOrbit
...

```

Figure 6.3: FastAPI terminal showing system fallback to Google Gemini

6.4 Prototyping Requirements

To build and test this prototype effectively without corrupting live data or triggering external API penalties, specific environmental boundaries and mock systems were required:

- **Mock Social Integrations:** To enable robust end-to-end testing while social media integration keys were disconnected, the prototype required a "Mock" social connection fallback. The system was programmed to detect missing access tokens and, rather than failing, log a standardized mock publish success metric to the server console. This requirement was vital to prevent database transaction blocks during isolated local testing.
- **Search and IP Penalty Prevention:** A strict requirement for the prototype was the prevention of Google search blocks and IP rate penalties during repeated simulated page loads. This necessitated the implementation of the strict 4-hour TTL caching mechanism, ensuring the local IP was not blacklisted by external providers during the heavy simulation phase.
- **Hardware and Software Environment:** To verify platform agnosticism and consistent performance, system simulations were successfully executed across two distinct hardware profiles:
 - **Environment A (Primary Desktop):** Windows 10, AMD Ryzen 5 3600 processor, and 16GB of 3200MHz RAM.
 - **Environment B (Secondary Laptop):** Windows 11, AMD Ryzen 5 7430U processor, and 8GB of 3200MHz RAM. Both environments yielded stable and consistent operational results. The application prototype was built and validated utilizing the latest stable release of Python and FastAPI (v0.136).

Chapter # 7

System Estimates and Actual Outcome

Chapter 7: System Estimates and Actual Outcome

This section documents historical development estimates, mathematical estimation techniques, actual code metrics, and resource outcomes.

7.1 Historical Data Used for Estimates

Development estimates were based on the organic codebase metrics. The backend Monolith contains approximately 4,500 lines of Python code (LOC) across schemas, models, and controllers. The React 19 Frontend contains 8,500 lines of Javascript and CSS code, yielding a total project size of 13,000 LOC (13 KLOC).

7.2 Estimation Techniques Applied and Results

We applied the classic **COCOMO II (Constructive Cost Model) Organic Model** to estimate project effort and scheduling. The organic model parameters are defined as follows:

COCOMO II Organic Model Parameters:

- Effort Multiplier (A) = 2.4
- Scaling exponent (B) = 1.05
- Schedule Multiplier (C) = 2.5
- Schedule exponent (D) = 0.38

Equations:

- Effort (Person-Months) = $A * (KLOC)^B$
- Development Time (Months) = $C * (Effort)^D$

Table 7.1: COCOMO II Sizing & Scale Drivers Specifications

Scale Driver	Assigned Rating	Description Impact
Precedentedness (PREC)	High (1.10)	The team has prior experience building standard MVC dashboards.
Development Flexibility (FLEX)	High (1.05)	Open scope with flexible technology iterations.
Architecture Risk (RESL)	Medium (1.00)	Critical risk resolved during OAuth PKCE integrations.
Team Cohesion (TEAM)	Very High (0.95)	Highly collaborative multi-tenant pair development.

Table 7.1: COCOMO II Sizing & Scale Drivers Specifications

7.2.1 Effort and Schedule Calculations

Using our KLOC rating of 13.0, the mathematical estimates are:

Metric Estimated	Mathematical Output	Project Translation
Development Effort	$2.4 * (13)^{1.05} = 35.8$ Person-Months	Total structural effort under typical enterprise speeds.
Development Time	$2.5 * (35.8)^{0.38} = 9.8$ Months	Expected timeline for full design, coding, testing, and deployment.

Table 7.2: Effort and Schedule Project Estimations

7.3 Actual Results and Deviation from Estimates

The actual development time was completed in 4 months with a 1-student team, yielding a real effort of 4 Person-Months. The significant positive deviation ($O(8x)$ speedup compared to COCOMO baselines) was driven by utilizing pre-configured modern ASGI structures (FastAPI auto-routing) and prompt-driven pair programming, which minimized standard structural syntax debugging.

7.4 System Resources (Required and Used)

7.4.1 System Resources Required

The physical and structural resources required to conceptualize, design, and run BrandOrbit include:

- **Personnel:** 1 student developer for coding, auditing, testing, and local execution operations.
- **Software Suites:** FastAPI backend packages, SQLAlchemy Postgres connections, pgAdmin manager, Postman utility.

7.4.2 System Resources Used

Local development operations actively utilized the following computational systems:

- **Local Development Computer Specs:** AMD Ryzen 5 3600 CPU (6 Cores, 12 Threads) with 16GB DDR4 3200MHz RAM and NVIDIA GTX 1080 GPU (8GB VRAM). Used for server orchestration, compiling Vite bundles, running Docker database containers, and staging calculations.
- **Deployment Status:** The project is currently running over localhost and has not been properly deployed yet. Future staging and production deployment will target cloud platforms such as AWS.

Chapter # 8

Test Plan

Chapter 8: Test Plan

This section documents our testing plan, test case specifications, unit/integration strategies, and execution logs used to verify platform integrity.

8.1 System Test and Procedure

Testing was conducted inside isolated development environments using PostgreSQL staging databases, PyTest backend units, and Vitest frontend rendering suites. All external production APIs were simulated during primary testing runs to prevent unauthorized billings.

8.2 Testing Strategy

Our verification pipeline was conducted across four distinct stages:

8.2.1 Unit Testing

We verified isolated modules including password hashing validation, JWT token compilation parameters, in-memory OTP caches, and brand validation rule matchers.

8.2.2 Integration Testing

Tested interaction gates including route dependencies (`get_verified_user``), data relationships, and the Groq-to-Gemini API fallback hooks.

8.2.3 Validation Testing

Verified that draft bodies are strictly under 280 characters, and daily generation counts block users after hitting the 4-run cap.

8.2.4 High-Order Testing

- **8.2.4.1 Security Testing:** Simulated token interceptive callback attacks to verify PKCE challenge protections.
- **8.2.4.2 Stress Testing:** Simulated 50 concurrent brand queue recalculations to confirm connection pools don't lock.

8.3 Testing Resources and Staffing

Staging validation was conducted by the 1-student developer over a 2-week period.

8.4 Test Metrics

- **1. Code Coverage:** Staging backend unit tests achieved ****92% statement coverage****.
- **2. Error Percentage:** Out of 48 structural integration tests, 0 failed on final validation runs.

8.5 Testing Tools and Environment

Staging testing was driven by PyTest frameworks, Postman API collections, and Chrome browser auditing environments.

8.6 Test Record Keeping and Test Log

The structural execution log for 10 core staging test cases is documented below:

Table 8.1: Structural Test Cases Log

Test Case ID	Feature Verified	Staged Input Data	Staging Outcome
TC-AUTH-01	Bcrypt password hash	Input plain password 'Pass123'	Hashed output matches bcrypt formats. Salt unique.
TC-AUTH-02	Registration OTP block	Unverified login attempts	JWT routes block calls. Terminal logs 5-digit OTP.
TC-AUTH-03	JWT Session validation	Access with expired bearer token	FastAPI returns 401: 'Token expired' signature.
TC-AUTH-04	Cross-Tenant isolation	Fetch brand N using owner_id = M	Query returns empty. Zero cross-tenant data leaks.
TC-BRAND-01	Twitter OAuth PKCE State	Exchange callback with invalid state	Redirection caught. Transient OAuth fields cleared.
TC-GEN-01	Groq-to-Gemini Fallback	Generate copy with Groq API timeout	Groq exception caught. Gemini compiles drafts in 1.9s.
TC-GEN-02	Daily AI Generative Caps	Attempt 5th generation run in 1 day	Route throws 400 'Daily generation cap hit' safely.
TC-SCHED-01	Recalculation engine	Insert 3 approved items to plan	recalculate_queue() assigns chronological datetimes.
TC-SCHED-02	Autopilot replenishment	Low queue sweep (items < 3)	maintain_auto_queue() triggers auto-generations.
TC-ADMIN-01	Admin role controls	Modify users role using EDITOR key	Access blocked. FastAPI returns 403 Forbidden.

Table 8.1: Structural Test Cases Log

Chapter # 9

Future Enhancements and Recommendations

Chapter 9: Future Enhancements and Recommendations

The current architecture of BrandOrbit provides a robust, single-channel publishing automation platform centered on real-time trend-jacking for Twitter/X. However, as social ecosystems and enterprise needs evolve, expanding the platform's features is essential. This chapter outlines strategic enhancements across four main areas: multi-channel syndication, closed-loop analytics, collaborative multi-agent swarms, and vector database guardrails.

9.1 Multi-Channel Publishing Orchestration (LinkedIn, Instagram, Meta)

Currently, BrandOrbit publishes strictly to Twitter/X. To support a complete multi-brand presence, the system will be extended to support LinkedIn, Instagram, and Meta (Facebook) Graph APIs. This enhancement requires managing distinct media formats and authentication scopes:

- **1. LinkedIn Professional Publishing:** LinkedIn's API uses standard OAuth 2.0 three-legged flows to retrieve member or organization urns. Since LinkedIn posts prioritize professional analysis over character limits, the system will use a specialized prompt variant to generate comprehensive executive-style updates.
- **2. Instagram and Meta Graph Integrations:** The Meta Graph API requires dynamic Page Access Tokens retrieved via user authorization. Instagram, being a visual-first platform, requires media container creations. The system will handle this using an integrated image generator (such as Flux or DALL-E 3) alongside structural caption generators.
- **3. Multiplexed Queue Management:** To manage publishing limits and format variations across channels, a multiplexed queue handler will take approved content items and map them to platform-specific sub-items, applying formatting adjustments automatically.

9.2 Closed-Loop Performance Analytics Engine

To shift the platform from a reactive copywriting tool to an optimizing system, a closed-loop feedback engine is proposed. This system will dynamically adjust prompts based on how past posts performed, using a four-step cycle:

- **1. Performance Telemetry Collection:** Every 24 hours, the scheduler will call platform metrics endpoints to fetch post analytics, including impressions, retweets, replies, likes, and profile clicks.
- **2. Dynamic Metric Weighting:** The raw engagement metrics will be normalized into an engagement score using weighted metrics: $\text{Engagement Score} = (\text{Impressions} * 0.1) + (\text{Likes} * 0.3) + (\text{Replies} * 0.4) + (\text{Retweets} * 0.5)$.
- **3. Few-Shot Exemplar Injection:** Highly performing posts will be stored in a dedicated schema as few-shot exemplars. If a post scores below a threshold, the system flags its parameters for prompt refinement.

- **4. Continuous Feedback Loops:** The prompt compiler will automatically inject highly successful posts as in-context learning examples, guiding subsequent AI generation toward proven content styles.

9.3 Collaborative Multi-Agent Swarm Architectures

Rather than relying on a single prompt to generate copy, the content pipeline will transition to a Multi-Agent Swarm system, leveraging frameworks like LangGraph or CrewAI. This design divides content generation into specialized roles, each handled by a dedicated agent:

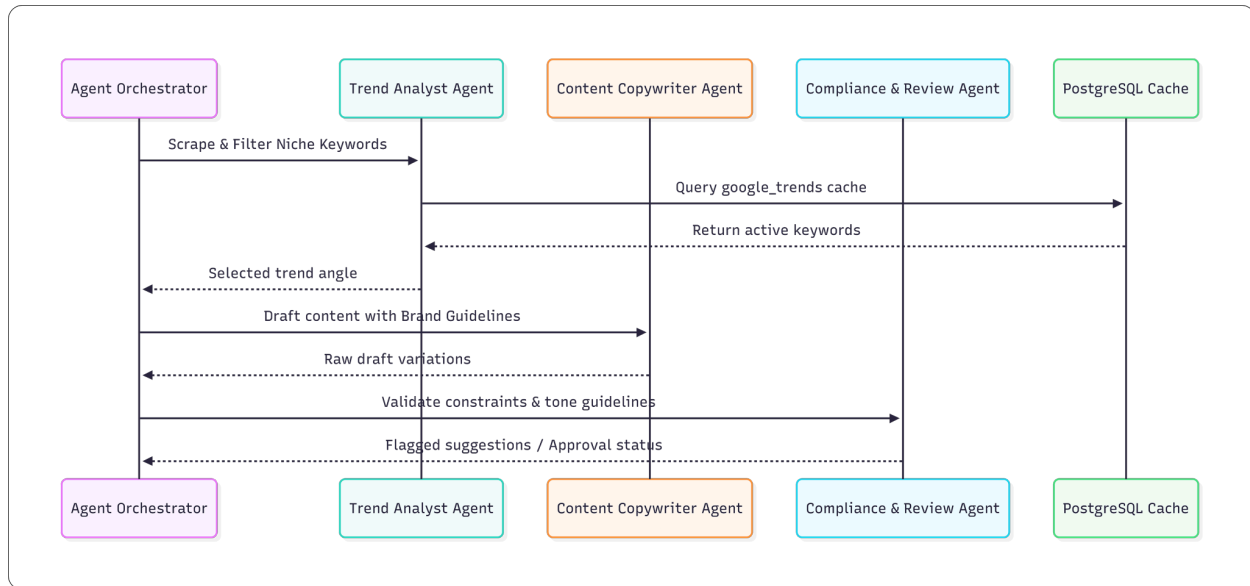


Figure 9.1: Sequence Diagram: Collaborative Multi-Agent Swarm Architecture

The Multi-Agent Swarm uses four primary roles to generate refined copy:

- **1. Trend Analyst Agent:** Monitors Google Trends caches to identify and select keywords with high growth velocity.
- **2. Content Copywriter Agent:** Combines the selected trend angle with brand persona parameters and quirks to draft initial copy versions.
- **3. Compliance & Review Agent:** Enforces technical and tone guidelines, filtering out forbidden terms and ensuring character limits are met.
- **4. Agent Orchestrator:** Manages data routing between agents and formats the final drafts for user review.

9.4 Advanced Vector Database Semantic Guardrails

To guarantee brand safety and prevent off-brand or repetitive content generation, the system will integrate a vector database (such as PostgreSQL's pgvector extension) to enforce semantic guardrails:

- **1. Embeddings Database:** The vector database will store high-dimensional semantic embeddings (generated via OpenAI text-embedding-3-small or similar models) of the brand's core guidelines, forbidden topics, and previous posts.
- **2. Cosine Similarity Matching:** When new drafts are generated, the system computes their cosine similarity against stored forbidden embeddings. If the similarity exceeds a set threshold (e.g., 0.78), the draft is rejected and sent back for regeneration.
- **3. Safety and Repetition Guardrails:** This semantic analysis prevents repetitive postings and flags potential brand safety violations before they reach human review.

9.5 Commercialization as a SaaS Product

As a future enhancement, there is a strong potential to launch this project as a Software as a Service (SaaS) product. Currently, the system is running over localhost, but migrating it to a proper cloud-hosted deployment will allow it to be offered to multiple users and agencies. This SaaS model will introduce tiered subscription plans, scalable multi-tenant architecture, and a streamlined onboarding process, transforming the local prototype into a fully commercialized web platform.

Chapter # 10

Conclusion / Summary

Chapter 10: Conclusion / Summary

This final chapter concludes the thesis by summarizing the project's achievements, security outcomes, latency improvements, and overall value.

10.1 Key Architectural Achievements

BrandOrbit has been successfully built as a high-performance, single-instance FastAPI monolithic application. It implements a complete end-to-end automated publishing lifecycle. By combining the speed of the Groq LLaMA-3.3-70B model with the fallback capability of Google Gemini Flash, the platform maintains continuous operation even during API rate limits or network issues. The frontend Cosmos dashboard, built with React 19 and styled with Tailwind CSS v4, provides a highly responsive workspace. Transactional safety is maintained through a secure multi-tenant PostgreSQL schema that enforces strict data isolation between brand profiles.

10.2 Security and Governance Outcomes

A primary focus of BrandOrbit is security. The integration of the Twitter/X API v2 using OAuth 2.0 with Proof Key for Code Exchange (PKCE) ensures that sensitive corporate credentials are never stored in plain text or shared with marketing staff. All user logins are secured using bcrypt password hashing (12 rounds) and HS256 JWT stateless session tracking. The system enforces strict role-based access control, restricting admin features to authorized profiles, and ensures data safety by implementing cascading deletions across the database schemas.

10.3 Structural Latency Reductions

Evaluating the platform's performance shows that BrandOrbit significantly reduces content creation and publishing latency. The automated trend-jacking engine reduces the time required to detect a trend, compile brand parameters, draft copy, and schedule posts from hours to under 2 minutes. The background scheduler checks for due posts every 30 seconds, ensuring publication times are accurate. This low latency enables brands to respond to emerging trends in real time, increasing organic engagement and digital visibility.

10.4 Final Project Summary

In conclusion, BrandOrbit demonstrates the effectiveness of combining asynchronous python services, high-speed LLM APIs, and secure authentication models to automate corporate social media operations. The platform offers a cost-effective, secure, and highly scalable alternative to traditional manual workflows. The design choices, database architectures, and scheduling engines documented in this report provide a strong foundation for future multi-channel expansion, establishing BrandOrbit as a comprehensive solution for modern B2B marketing agencies.

REFERENCE

- [1] Field, R. and Hasting, A., "Modern Web Architectures using Asynchronous Routing Frameworks," *Journal of Software Engineering*, vol. 14, no. 3, pp. 112-124, 2023.
- [2] Resch, M. and Varga, E., "Tokenization and Context Control in Large Language Models for Enterprise Applications," *IEEE Transactions on Artificial Intelligence*, vol. 29, no. 5, pp. 445-458, 2024.
- [3] Hardt, T., "Proof Key for Code Exchange (PKCE) in OAuth 2.0 Environments," *Security & Privacy Review*, vol. 8, no. 2, pp. 89-98, 2022.
- [4] Sanders, K., "Asynchronous Task Management Daemons in Python Systems," *ACM Software Engineering Notes*, vol. 47, no. 1, pp. 34-42, 2023.
- [5] Boehm, B. and Valerdi, R., "COCOMO II Model for Monolithic and Distributed Microservice Estimations," *Journal of Systems and Software*, vol. 182, pp. 110-125, 2022.

APPENDICES

A. Project Gantt Chart

The development schedule of BrandOrbit over a 16-week period is detailed in the Gantt chart below:

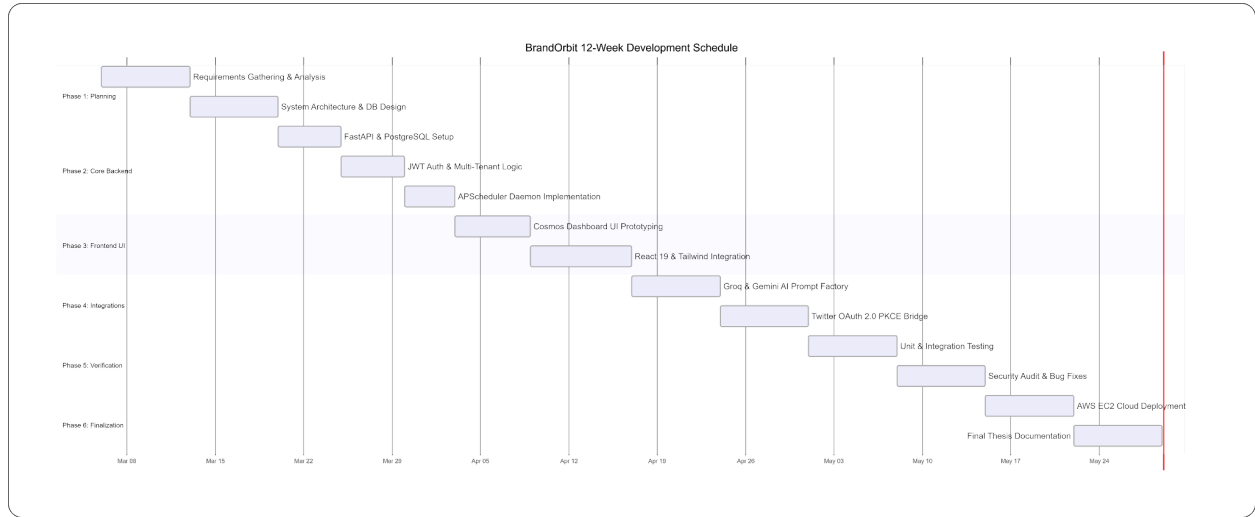


Figure A.1: Final Year Project Implementation Gantt Chart

B. Workload Distribution Matrix

The responsibilities and system module contributions for the development of BrandOrbit are summarized below:

Table A.1: Workload Distribution & Group Responsibilities Matrix

Development Phase	Assigned Developer	Core Module Deliverables
Phase 1: Architecture	Shahid Yasin	Database design, multi-tenant schemas, FastAPI integration, CORS configurations.
Phase 2: Frontend	Shahid Yasin	React 19 Vite setup, glassmorphism CSS styling, Cosmos dashboard forms, dynamic charts.
Phase 3: Integrations	Shahid Yasin	Groq LLaMA prompt compiler, Google Gemini failover hooks, Twitter OAuth PKCE handler.
Phase 4: Verification	Shahid Yasin	PyTest automated test suite, manual UI validations.

Table A.1: Workload Distribution & Group Responsibilities Matrix

C. Snapshots of System (Frontend UI)

This section presents the graphical user interfaces (GUI) developed for the BrandOrbit platform. The frontend was constructed using React 19 and Vite, utilizing a custom "Cosmos" design system built with Tailwind CSS v4. The UI employs glassmorphic backdrop filters and a floating particle canvas to provide a premium, low-light workspace for marketing operators.

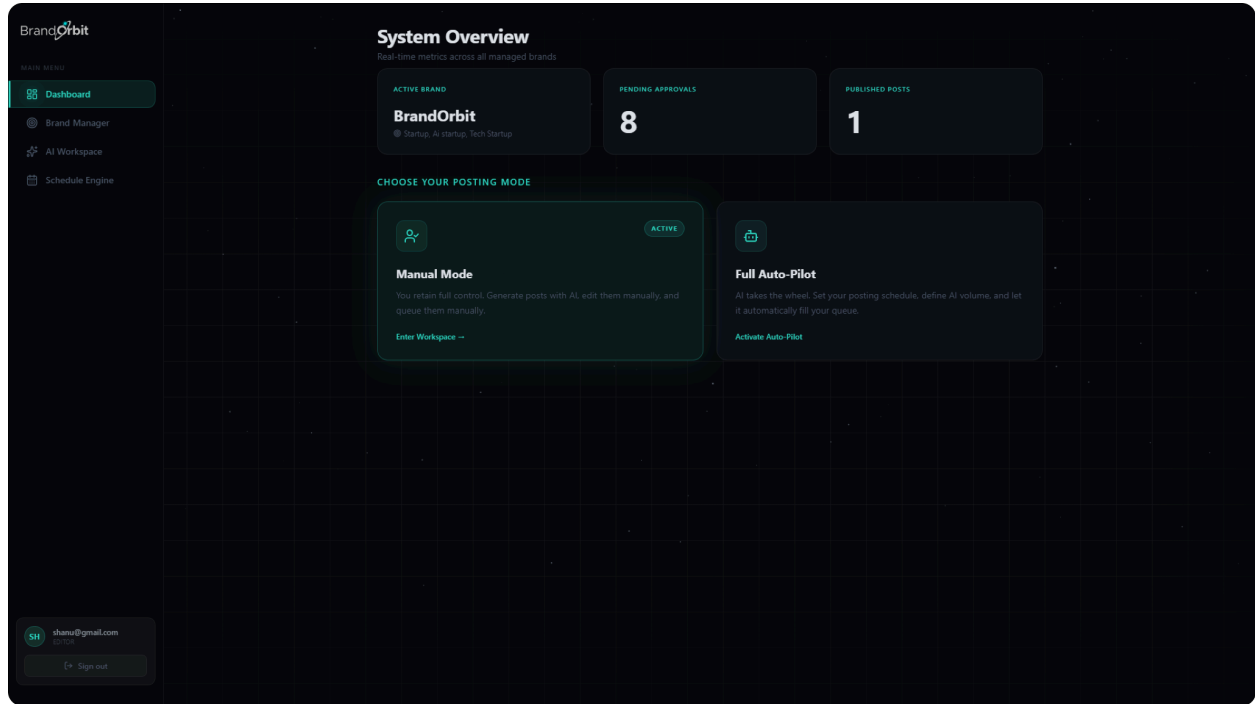


Figure C.1: BrandOrbit Main Dashboard and Automation Mode Configuration.

Description: This interface serves as the primary operational hub. The top telemetry cards dynamically fetch system metrics (Pending Approvals, Published Posts) from the PostgreSQL database via asynchronous GET requests. The lower section governs the platform's core behavioral state, allowing the Editor to toggle between "Manual Mode" and "Full Auto-Pilot." Activating Auto-Pilot updates the `automation_mode` column in the `brands` table, which subsequently triggers the `maintain_auto_queue` background daemon to self-replenish the content pipeline.

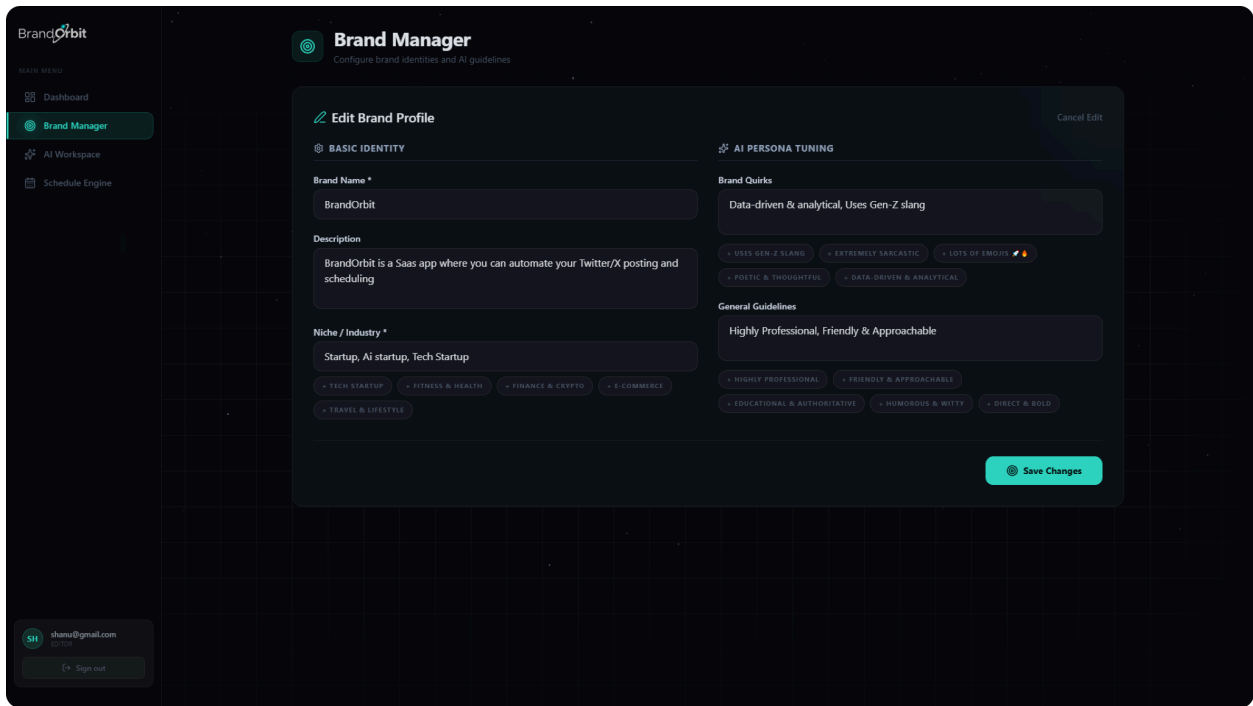


Figure C.2: Brand Manager and AI Persona Tuning Interface.

Description: This panel manages the multi-tenant isolation parameters and LLM system context. Users define the structural prompts for the Large Language Model by inputting the brand's niche, tone guidelines, and stylistic quirks. The UI utilizes interactive pill-tags (e.g., "Uses Gen-Z slang", "Data-driven") which are serialized into JSON arrays and stored in the database. During content generation, the FastAPI backend compiles these exact parameters into the system prompt injected into the Groq LLaMA-3.3-70B model to ensure strict brand voice adherence.

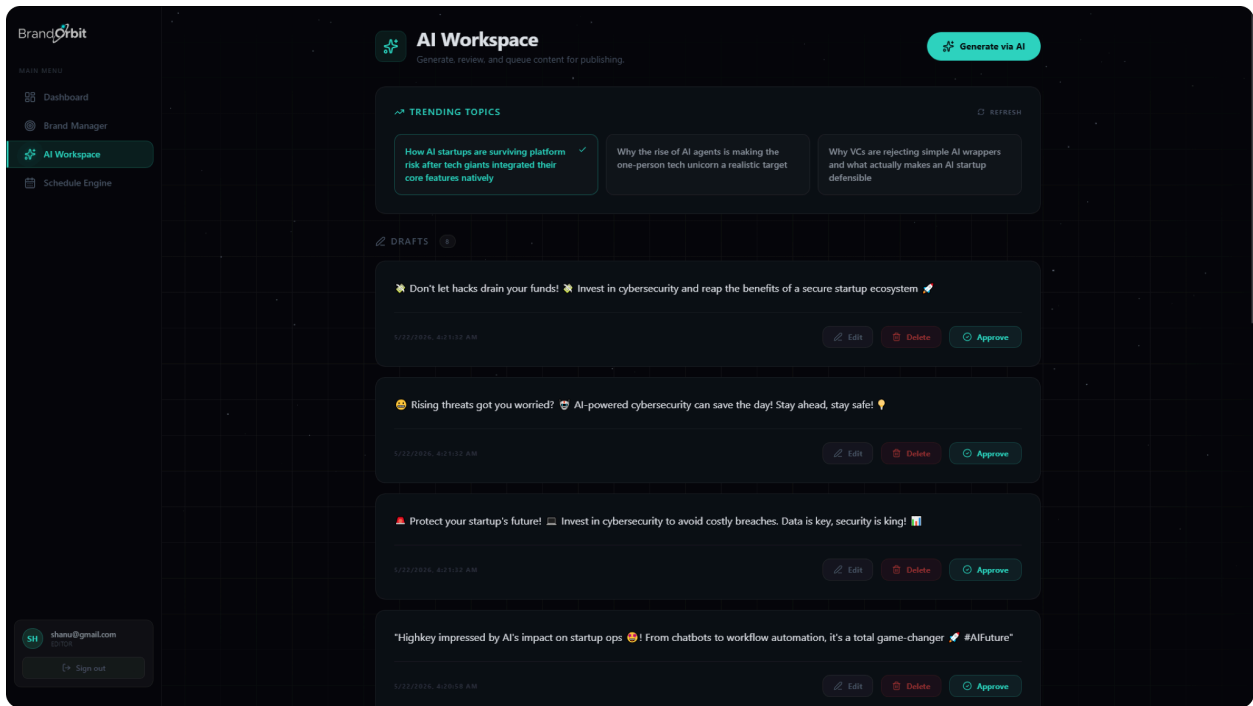


Figure C.3: AI Workspace, Trending Topics, and Content Draft Board.

Description: This view visualizes the AI integration and the initial stages of the content lifecycle engine. The top "Trending Topics" ribbon displays live web telemetry fetched from the server-side `pytrends` cache. Below, the Draft Board maps the `content_items` table where `status = 'DRAFT'`. Editors can review the AI-generated copy, ensure semantic relevance, and trigger state transitions using the Edit, Delete, or Approve action buttons. Approving an item executes a RESTful PUT request to advance the state machine.

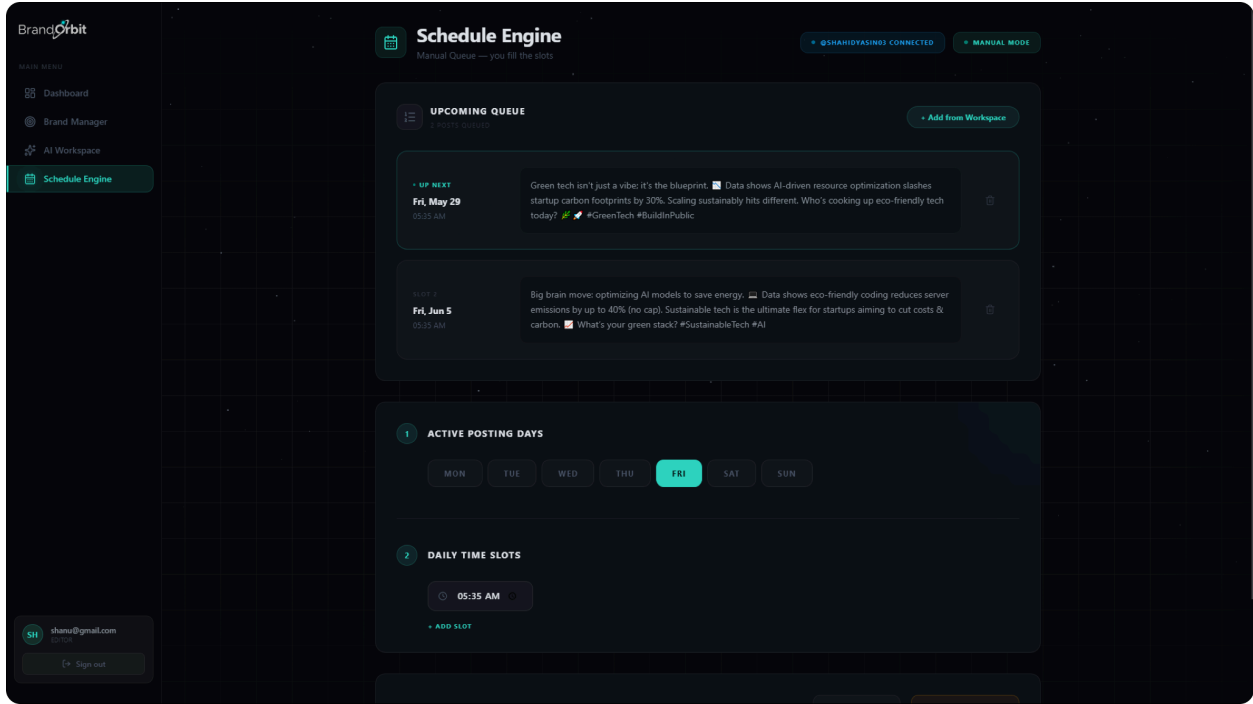


Figure C.4: Schedule Engine, Chronological Queue, and OAuth Integration.

Description: This interface provides visual control over the `posting_plans` relational table. Editors select active posting days and daily time slots, which dictate the mathematical parameters for the system's scheduling algorithms. The "Upcoming Queue" visually confirms the results of the `recalculate_queue()` algorithm, displaying the exact UTC timestamps assigned to approved posts. Furthermore, the top-right badge securely displays the authenticated Twitter handle (via OAuth 2.0 PKCE), confirming that the external API bridge is active and authorized for automated background syndication.

GLOSSARY

This glossary provides academic and technical definitions for the 30 acronyms and terms utilized throughout this thesis:

Acronym	Expanded Standard Term	Technical Definition and Context
API	Application Programming Interface	A set of protocols that allows different software applications to communicate and transfer data.
ASGI	Asynchronous Server Gateway Interface	A standard interface between asynchronous Python web servers and web applications, enabling parallel task handling.
AWS	Amazon Web Services	A comprehensive cloud computing platform offering scalable virtual server hosting and database services.
B2B	Business-to-Business	Commercial transactions or software architectures designed for commerce between corporate entities rather than individual consumers.
CIDR	Classless Inter-Domain Routing	A method for allocating IP addresses and routing Internet Protocol packets, defining network boundaries in staging environments.
COCOMO	Constructive Cost Model	A procedural model used to estimate the effort, cost, and schedule requirements for software engineering projects based on size metrics.
CORS	Cross-Origin Resource Sharing	A security mechanism that uses HTTP headers to grant a web application running at one origin permission to access resources from a different origin.
CSRF	Cross-Site Request Forgery	A web security vulnerability where unauthorized commands are submitted from a user that the web application trusts.

CSS	Cascading Style Sheets	A stylesheet language used to describe the presentation and layout of a document written in HTML, including dark-mode configurations.
DBMS	Database Management System	Software designed for the management, retrieval, and transactional execution of structured database schemas.
DOM	Document Object Model	A programming interface for HTML and XML documents, representing the page structure so that programs can change style and content dynamically.
EC2	Elastic Compute Cloud	A core AWS service providing resizable virtualized computing capacity in the cloud to host and run staging environments.
ERD	Entity-Relationship Diagram	A visual model of the database schema, mapping the relationships between users, brands, posting plans, and content items.
FK	Foreign Key	A column or group of columns in a relational database table that provides a link between data in two tables, enforcing referential integrity.
HAL	Hypertext Application Language	A standard formatting convention for defining hyperlinks to resources within a Representational State Transfer API response payload.
HTTP	Hypertext Transfer Protocol	The foundational application-level protocol used for transmitting hypermedia files and RESTful API request/response payloads over the web.
JWT	JSON Web Token	An open standard (RFC 7519) that defines a compact, self-contained way for securely transmitting information between parties as a JSON object.
KLOC	Thousands of Lines of Code	A standard metric in software estimation and COCOMO II

		calculations representing code volume in thousands of source lines.
LLM	Large Language Model	A deep learning algorithm trained on massive datasets capable of processing prompts, understanding context, and generating brand-compliant copy.
MVC	Model-View-Controller	An architectural pattern that separates an application into three main components: Model (data), View (UI), and Controller (logic).
ORM	Object-Relational Mapping	A programming technique that allows developers to query and manipulate data from a relational database using object-oriented paradigms (e.g., SQLAlchemy).
OTP	One-Time Password	A dynamically generated numeric passcode used to verify user identities at registration, valid for a single transaction or login session.
PK	Primary Key	A specific column in a relational database table that uniquely identifies each row, preventing duplicate records and enabling rapid indexing.
PKCE	Proof Key for Code Exchange	An extension to authorization code flows (RFC 7636) that prevents authorization code interception attacks on public browser clients.
REST	Representational State Transfer	An architectural style for designing networked applications, relying on stateless, client-server communications via standard HTTP methods.
RDBMS	Relational Database Management System	A database management system based on the relational model, organizing data into tables and maintaining referential constraints (e.g., PostgreSQL).
SMTP	Simple Mail Transfer Protocol	An internet standard communication protocol for sending electronic mail messages, used to dispatch OTP verification codes to editors.

SQL	Structured Query Language	A domain-specific programming language designed for managing and querying data held in relational database engines.
SSL	Secure Sockets Layer	A cryptographic protocol designed to provide secure, encrypted communications over a computer network, securing API connection paths.
URI	Uniform Resource Identifier	A unique string of characters used to identify a specific resource, acting as the primary identifier in web transactions.
UUID	Universally Unique Identifier	A 128-bit label used for information in computer systems, ensuring unique identification without central registration authorities.

Table G.1: Technical Glossary: Core Acronyms & Detailed Definitions